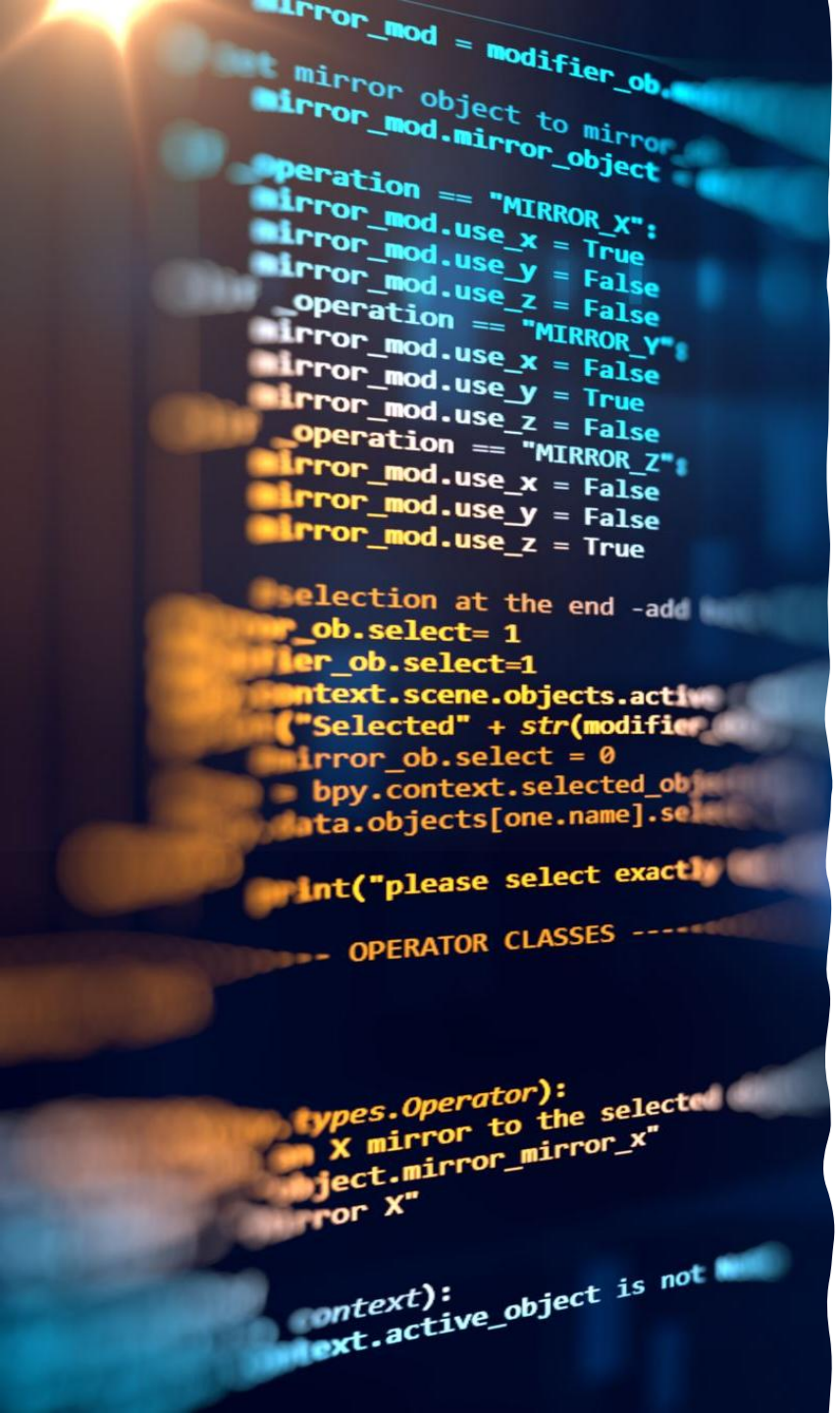


HTML and XHTML

Brief History of Web Technologies Contd…

The W3C is the main standards organization for the World Wide Web.

To promotes compatibility the W3C produces **recommendations**

(also called specifications).

In 1998, theW3C turned its attention to a new specification called XHTML1.0

XHTML 1.0 was a version of HTML that used stricter XML (Extensible Markup Language) syntax rules.

Markup Language

A markup language is simply a way of annotating a document in such a way as to make the annotations distinct from the text being annotated.

Example of markup languages: HTML, Tex, XML, and XHTML

Allow users to control how text and visual elements will be laid out and displayed

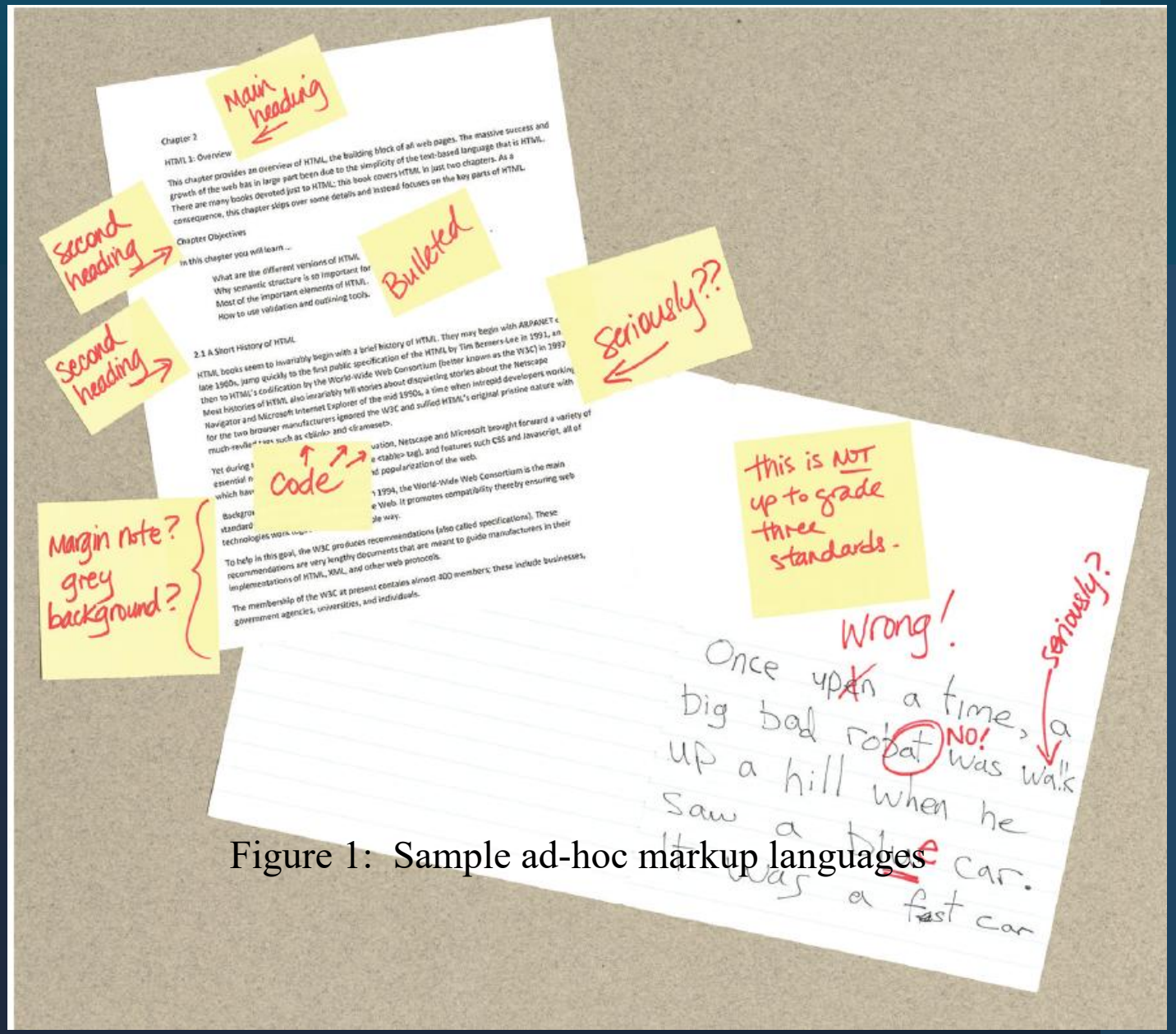


Figure 1: Sample ad-hoc markup languages

HTML (Hyper Text Markup language)

HTML is a markup language used to create Web pages.

HTML is developed and maintained by W3C

In HTML, the term hyper signifies the navigation from one location to another location in a non-linear fashion.

HTML defined the content; ie content and the layout of a web page with the help of elements and attributes

Versions of HTML

HTML 1.0(1992): Provides the idea of separation between the logical structure and presentational elements in a web page.

HTML 2.0(1995): Updated version of HTML 1.0 which includes new features such as file uploads.

HTML 3.0(1996): Updated version of HTML 2.0, includes support for including style sheets and has additional elements such as FIG, LISTS, LINK etc

HTML 4.0(1997): Refers to a version that has introduced new elements for style sheets , scripts , frames, emdded objects, complex tables.

HTML 5.0(2009): This version supports audio, video, more interactive web applications and websites

Aims of HTML 5

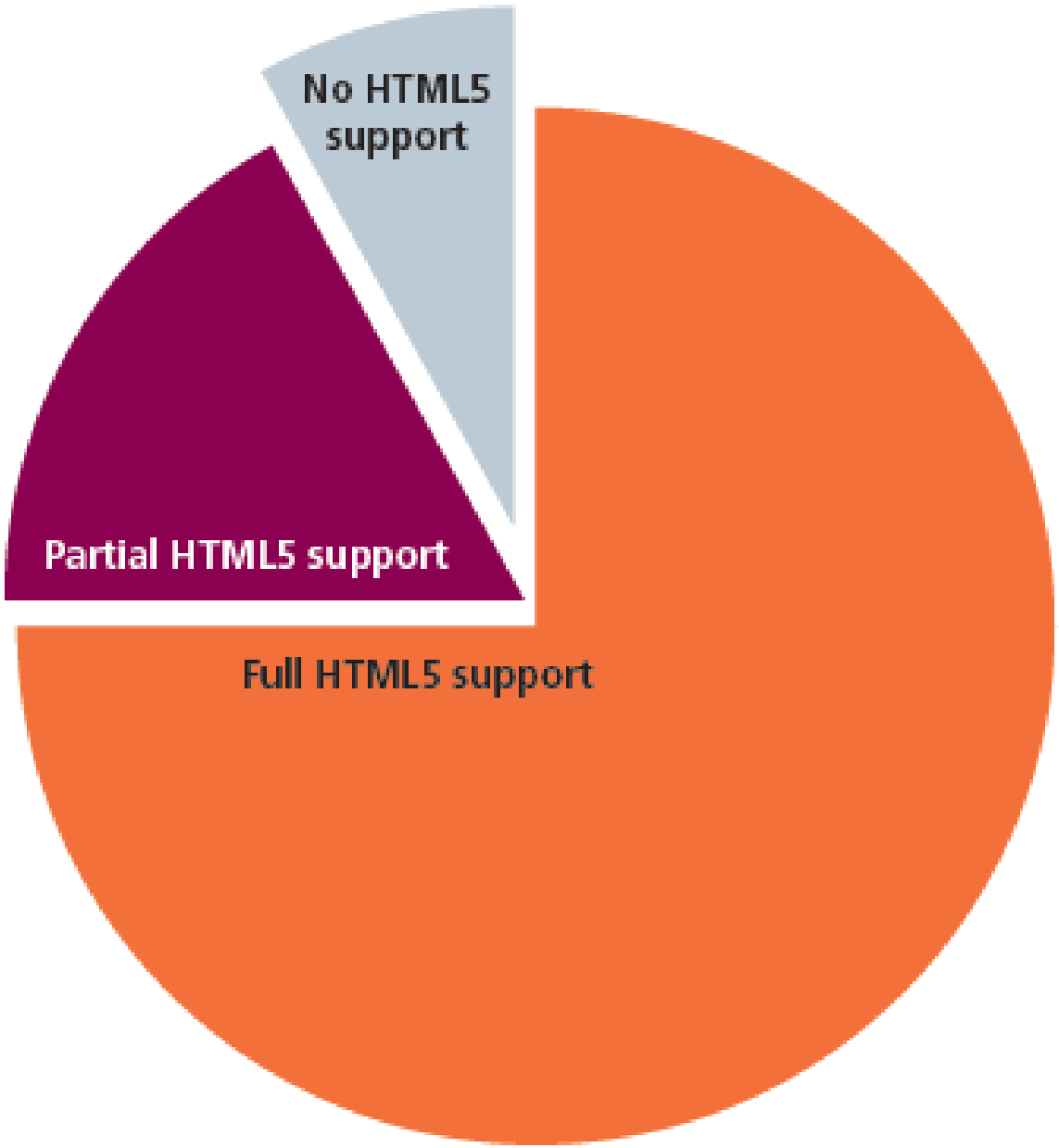
Specify unambiguously how browsers should deal with invalid markup.

- If a closing tag is missing or misplaced, the browser may infer where to close a tag or ignore extra closing tags

Provide an open, non-proprietary programming framework (via Javascript) for creating rich web applications

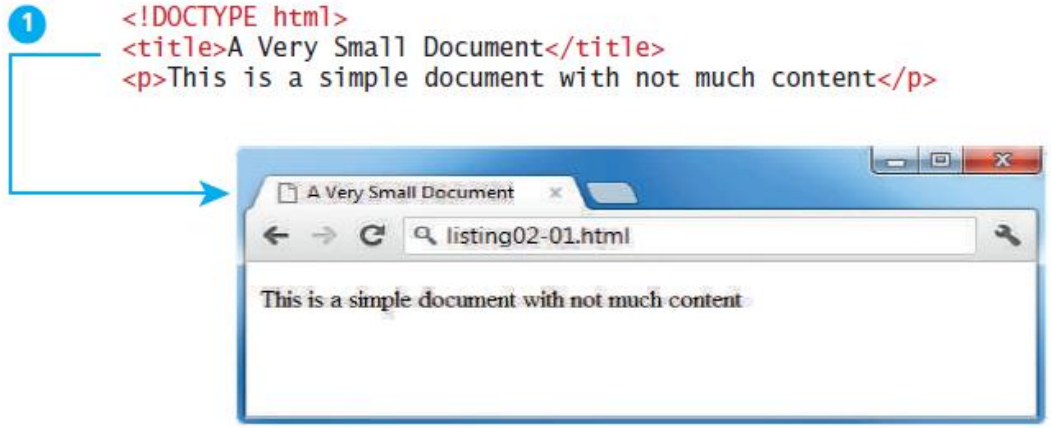
Be backwards compatible with the existing web.

Browser usage and HTML5 support



Structure of HTML Documents

Simple HTML forms



Simplest possible HTML Document

- DOCTYPE (Document Type Definition): which tells the browser what type of document it is about to process.
- All HTML documents must start with a document type declaration
- Title is used to provide a broad description of the content.
- The title is typically displayed by the browser in its window and/or tab.
- <p> is used to display the contents in the browser window.

HTML Syntax

Elements and attributes

- HTML documents are composed of textual content and HTML elements

- An HTML element can contain text, other elements, or be empty.

- It is identified in the HTML document by tags.

- An HTML element is defined by a start tag, some content, and an end tag:

  <tagname>Content goes here...</tagname>

- The fundamental syntactic units of HTML are called tags, defined between angular braces.

  Example: This is simple

 tag in HTML defines a paragraph.

HTML Syntax contd..



- HTML elements can also contain attributes.
- An **HTML attribute** is a name=value pair that provides more information about the HTML element

Simple HTML forms

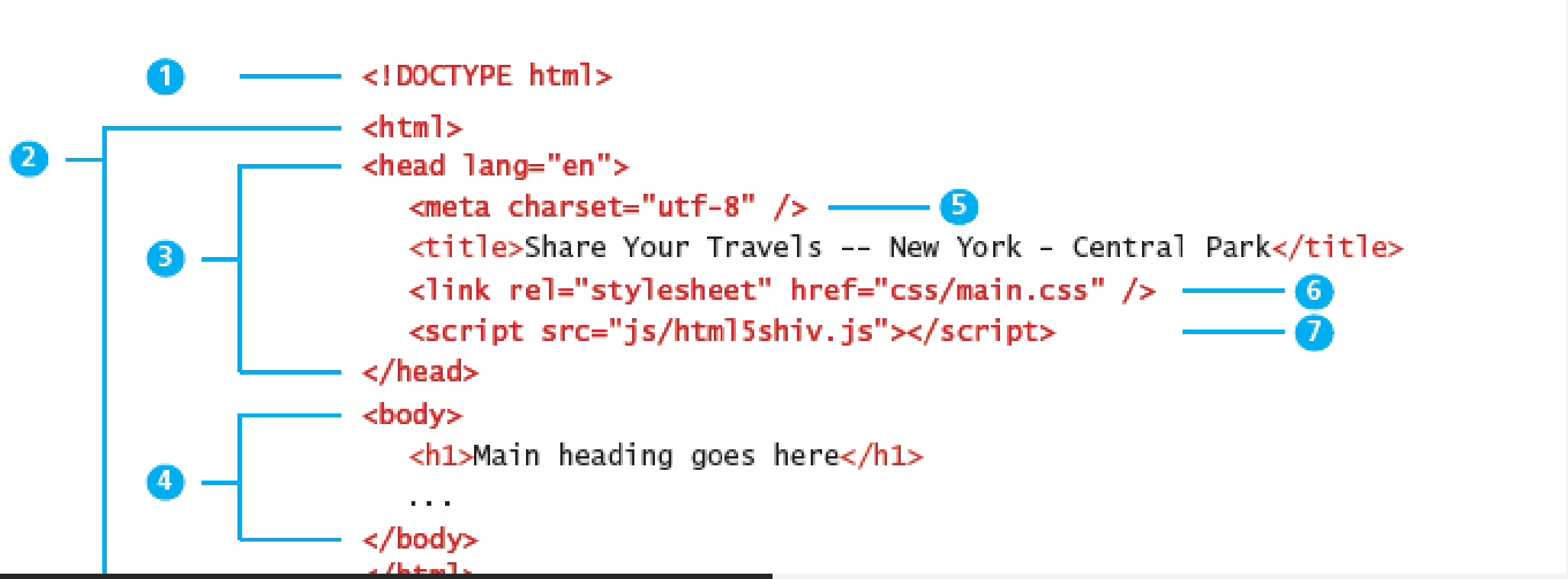
The <html> element is sometimes called the **root element** as it contains all the other HTML elements in the document.

HTML pages are divided into two sections: the **head** and the **body**

The <head> tag contains descriptive elements about the document, such as its title, any style sheets or JavaScript files it uses, and other types of meta information used by search engines and other programs

The <body> tag defines the **main content area** of a web page. Everything inside the <body> tag is what the browser displays as the visible content to users

```
<!DOCTYPE html>
  <head>
    <title>Sample Page</title>
  </head>
  <body>
    <p>Hello World</p>
  </body>
</html>
```



Simple HTML forms

- Lang is an optional attribute that tells the browser the natural language that is being used for textual content in the HTML document, which is English in this example.
- <meta> Element declares that the character encoding for the document is UTF-8

Simple HTML forms Contd..

Character encoding refers to which character set standard is being used to encode the characters in the document.

Item 6: an external CSS style sheet file that is used with this document.

Styles can also be defined within an HTML document, for consistency's sake, most sites place most or all of their style definitions within one or more external style sheet files.

Item 7: references an external JavaScript file

Nesting HTML Elements

Often an HTML element will contain other HTML elements.

In such a case, the container element is said to be a parent of the contained, or child, element.

Any elements contained within the child are said to be **descendants** of the parent element

Empty HTML Elements

```
<!DOCTYPE html>
<html>
<body>

<p>This is a <br> paragraph with a line break.</p>

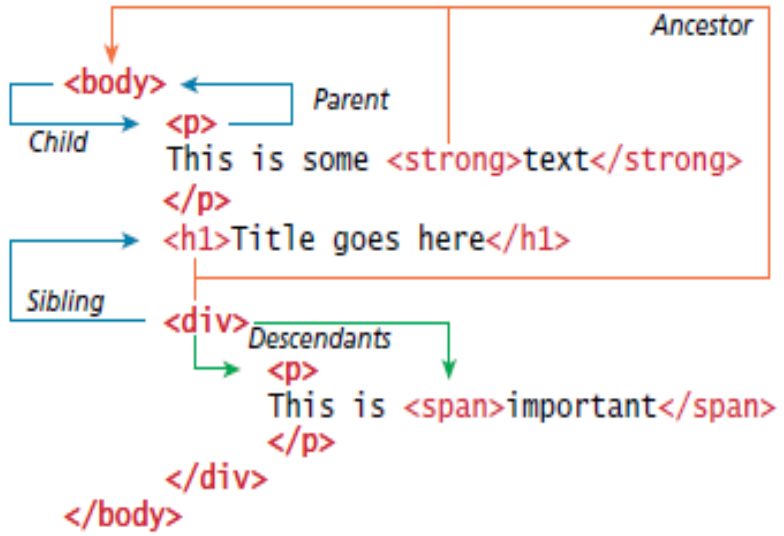
</body>
</html>
```

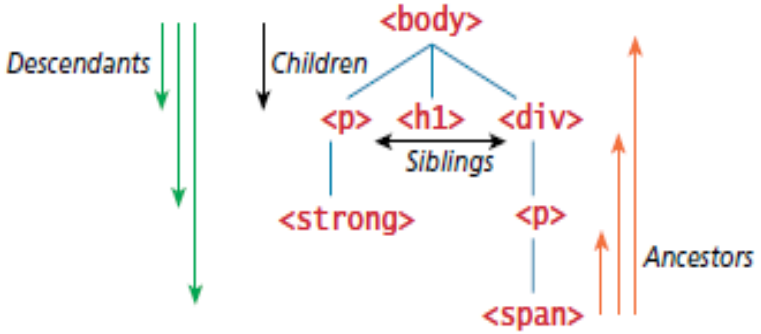
Output:

This is a
paragraph with a line break.

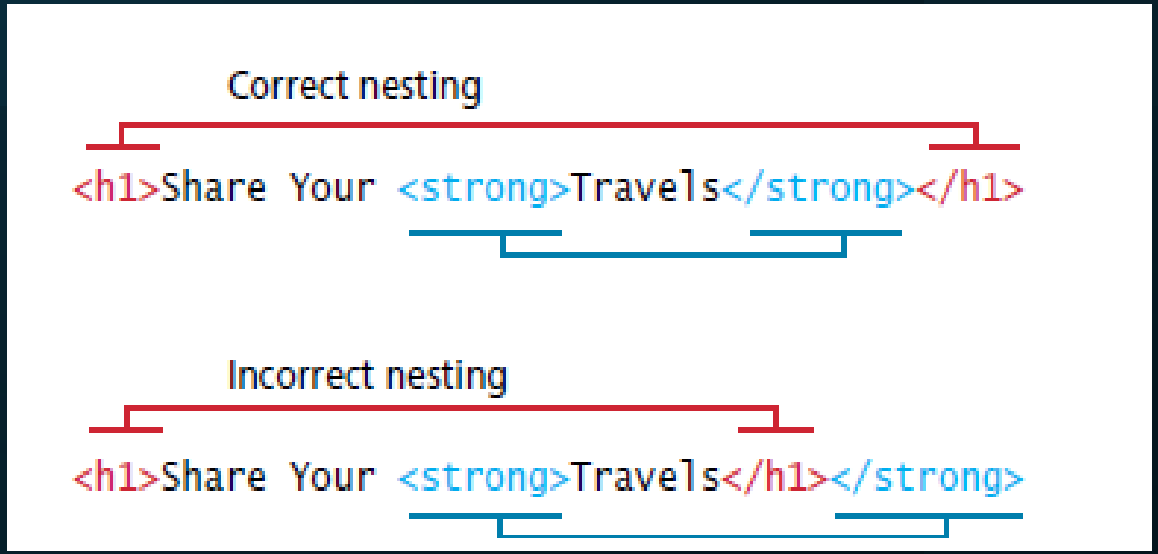
Nesting HTML Elements Contd…

HTML document outline



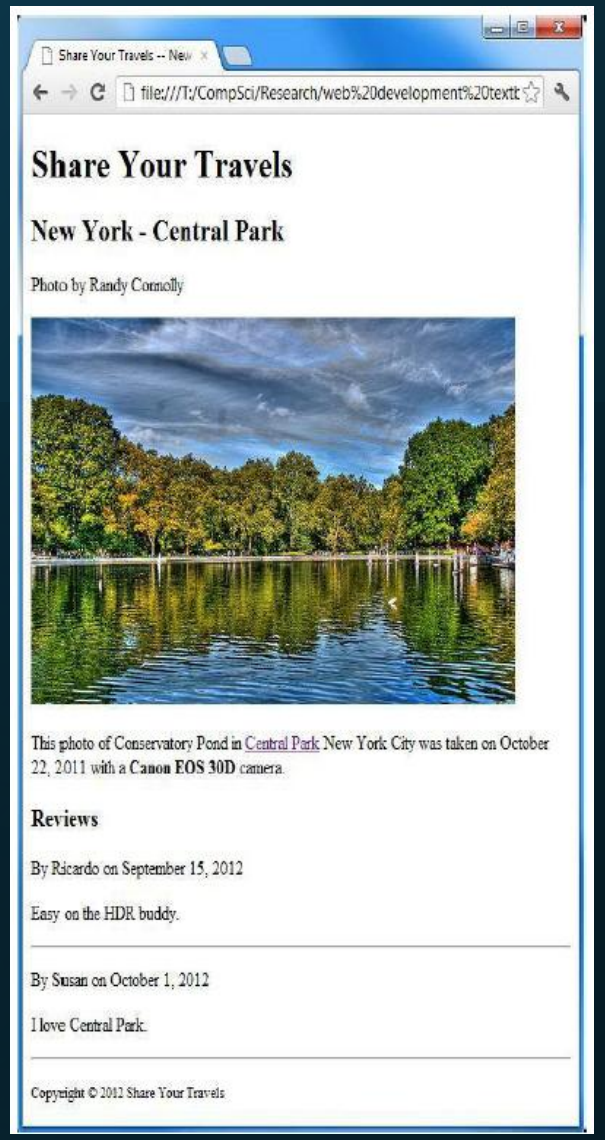


Nesting HTML Elements Contd…



Example of proper and improper nesting

HTML Elements




```
<body>
  <h1>Share Your Travels</h1>
  <h2>New York - Central Park</h2>
  <p>Photo by Randy Connolly</p>
  <p>This photo of Conservatory Pond in
    <a href="http://www.centralpark.com/">Central Park</a>
    New York City was taken on October 22, 2015 with a
    <strong>Canon EOS 30D</strong> camera.
  </p>
  <img src="images/central-park.jpg" alt="Central Park" />

  <h3>Reviews</h3>
  <div>
    <p>By Ricardo on <time>September 15, 2015</time></p>
    <p>Easy on the HDR buddy.</p>
  </div>

  <div>
    <p>By Susan on <time>October 1, 2015</time></p>
    <p>I love Central Park.</p>
  </div>

  <p><small>Copyright &copy; 2015 Share Your Travels</small></p>
</body>
```

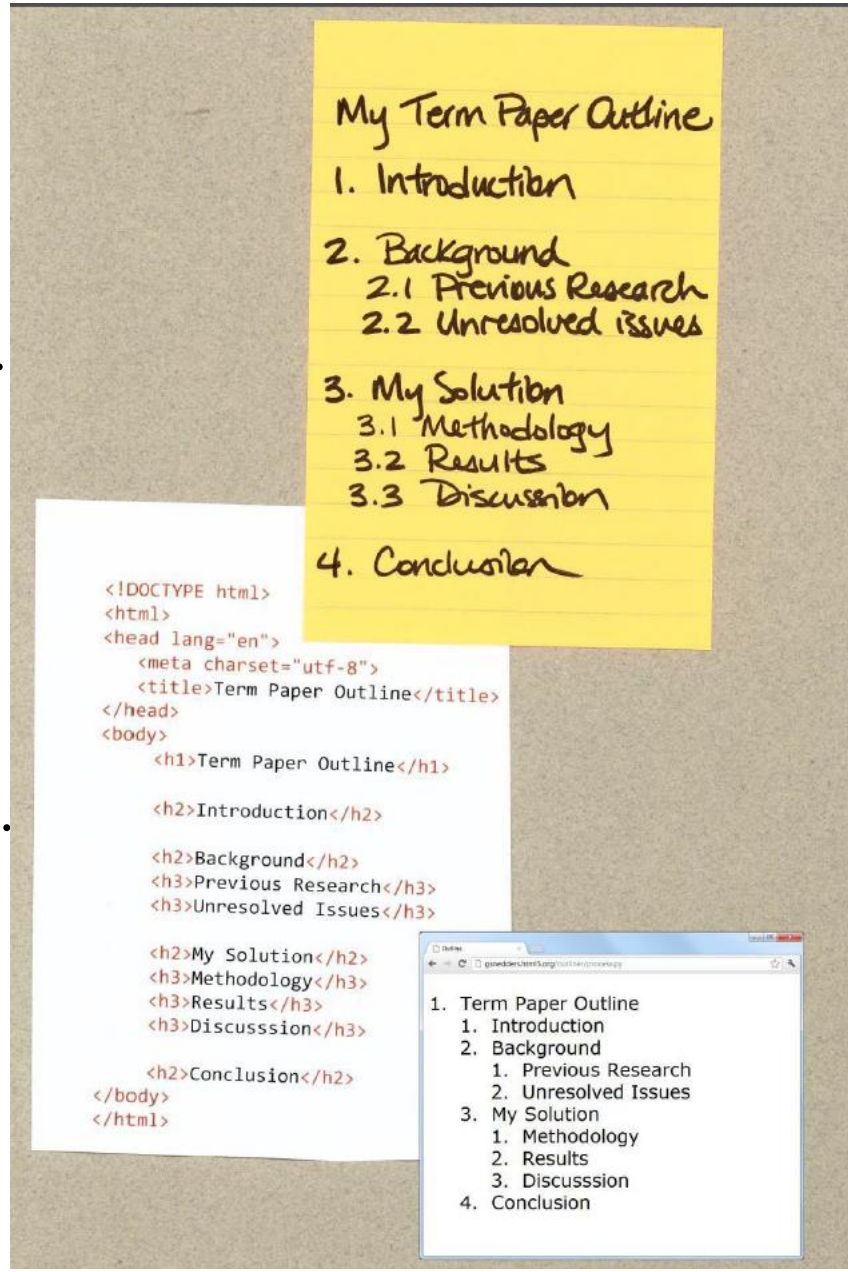
Headings

- HTML provides six levels of heading (**h1, h2, h3, …**), with the higher heading number indicating a heading of less importance.

- Headings are an essential way for document authors use to show their readers the structure of the document

- The browser has its own default styling for each heading level.

- However, these are easily modified and customized via CSS



Headings
Be semantically accurate

1

In practice, specify a heading level that is semantically accurate

2

Do not choose a heading level because of its default presentation

- e.g., choosing <h3> because you want your text to be bold and 16pt

3

Choose the heading level because it is appropriate

- e.g., choosing <h3> because it is a third level heading and not a primary or secondary heading

Bigger Headings

- Each HTML heading has a default size. However, you can specify the size for any heading with the style attribute, using the CSS font-size property:

- Example:

- <h1 style="font-size:60px;">Heading 1</h1>

Paragraphs

- ✓ Paragraphs are the most basic unit of text in an HTML document.
- ✓ Notice that the ** ** tag is a container and can contain HTML and other **inline HTML elements**

Example: This is a paragraph with bold text and a link

- ✓ Inline HTML elements refers to HTML elements that do not cause a paragraph break but are part of the regular “flow” of the text.
- ✓ The browser will automatically remove any extra spaces and lines when the page is displayed

HTML Horizontal Rules

The <hr> tag defines a thematic break in an HTML page, and is most often displayed as a horizontal rule.

The <hr> element is used to separate content (or define a change) in an HTML page:

```
<!DOCTYPE html>
<html>
   <body>
        <h1>This is heading 1</h1>
        <p>This is some text.</p>
     <hr>
       <h2>This is heading 2</h2>
      <p>This is some other text.</p>
    <hr>
      <h2>This is heading 2</h2>
     <p>This is some other text.</p>
</body>
</html>
```



What is the output?

```
<!DOCTYPE html>
<html>
<body>
<p>In HTML, spaces and new lines are ignored:</p>
<p>
  hello.
  hi.
  CCE.
  IT.
</p>
</body>
</html>
```

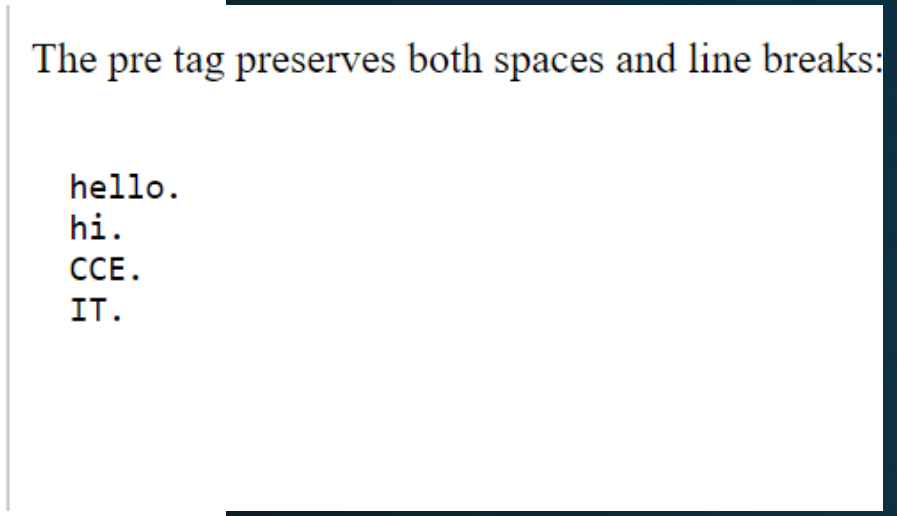
In HTML, spaces and new lines are ignored:

hello. hi. CCE. IT.

The HTML <pre> Element

The HTML <pre> element defines preformatted text.

```
<!DOCTYPE html>
<html>
<body>
<p>The pre tag preserves both spaces and line
breaks:</p>
<pre>
 hello
 . hi.
 CCE.
 IT.
</pre>
</body>
</html>
```



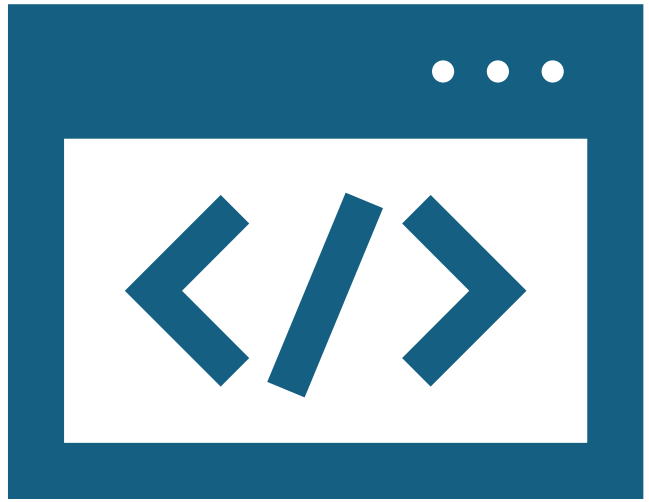
HTML Styles

- The HTML style attribute is used to add styles to an element, such as color, font, size, and more.
- The HTML style attribute has the following syntax:

<tagname style="property:value;">

- The property is a CSS property. The value is a CSS value.

- The CSS background-color property defines the background color for an HTML element.

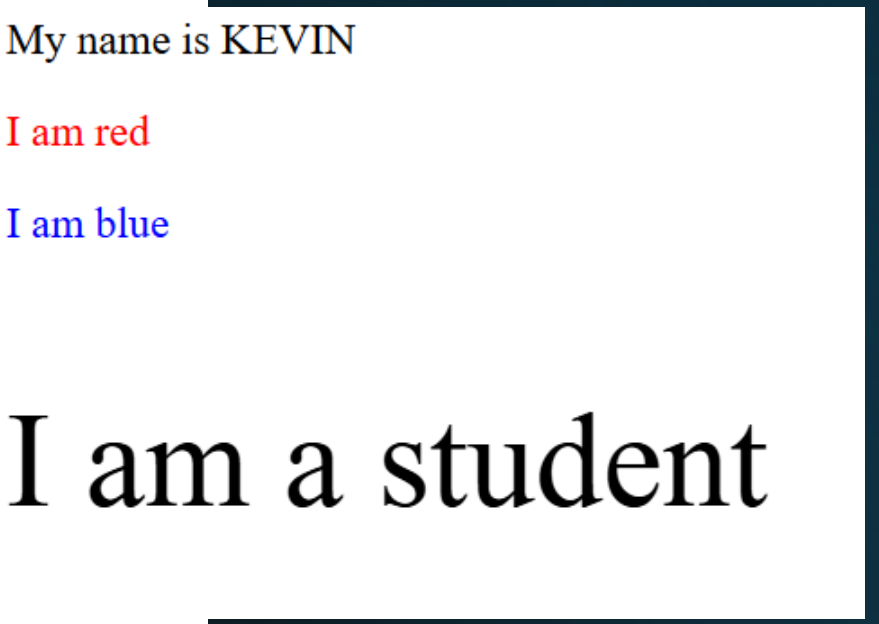


HTML Styles

```
<!DOCTYPE html>
<html>
<body>

<p>My name is KEVIN</p>
<p style="color:red;">I am red</p>
<p style="color:blue;">I am blue</p>
<p style="font-size:50px;">I am a student
</p>

</body>
</html>
```

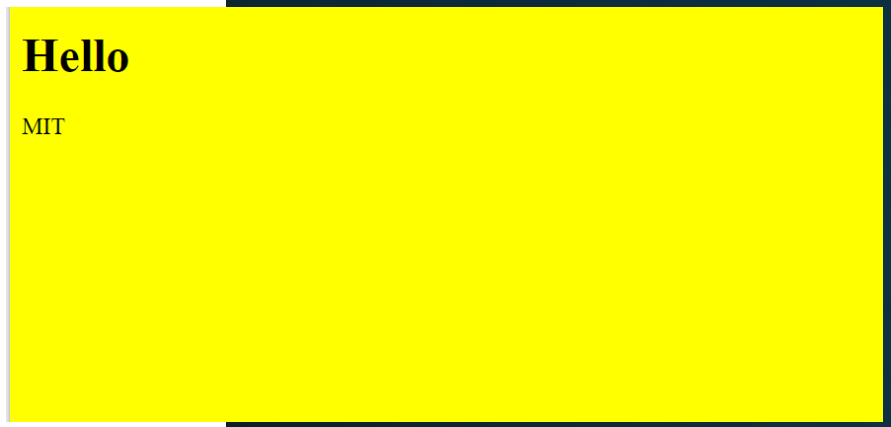


Background Color

```
<!DOCTYPE html>
<html>
<body style="background-color:yellow;">

<h1>Hello</h1>
<p>MIT</p>

</body>
</html>
```



Set background color for two different elements:

```
<!DOCTYPE html>
<html>
<body>

<h1 style="background color:blue;">hello</h1>
<p style="background-color:GREEN;">MIT</p>

</body>
</html>
```



Write the HTML

This is a heading

This is a paragraph.

```
<!DOCTYPE html>
<html>
<body>

<h1 style="color:blue;">This is a heading</h1>
<p style="color:red;">This is a paragraph</p>

</body>
</html>
```

Text Alignment

```
<!DOCTYPE html>
<html>
  <body>
   <h1>Hello</h1>
   <h1 style="text-align:center;">Hello</h1>
   <p style="text-align:center;">Hi MIT.</p>

  </body>
</html>
```



HTML Text Formatting

HTML contains several elements for defining text with a special meaning.

Formatting elements were designed to display special types of text:

- <b> - Bold text
- - Important text
- <i> - Italic text
- - Emphasized text
- <mark> - Marked text
- <small> - Smaller text
- <del> - Deleted text
- <ins> - Inserted text
- <sub> - Subscript text
- <sup> - Superscript text

Write the code for the following output

Type this in bold

Type this in italic

This is $_{\text{subscript}}$ and ^{superscript}

—

```
<!DOCTYPE html>
<html>
<body><p><b>Type this in bold</b></p>
<p><i>Type this in italic</i></p>
<p>This is<sub> subscript</sub> and
<sup>superscript</sup></p>

</body>
</html>
```

HTML <mark> Element

```
<!DOCTYPE html>
<html>
<body>

<p>Hello <mark>MIT</mark> Hiii!!!!!.</p>

</body>
</html>
```

Hello MIT Hiii!!!!!.

HTML <del> Element

goodmorning
~~Good evening~~
good night

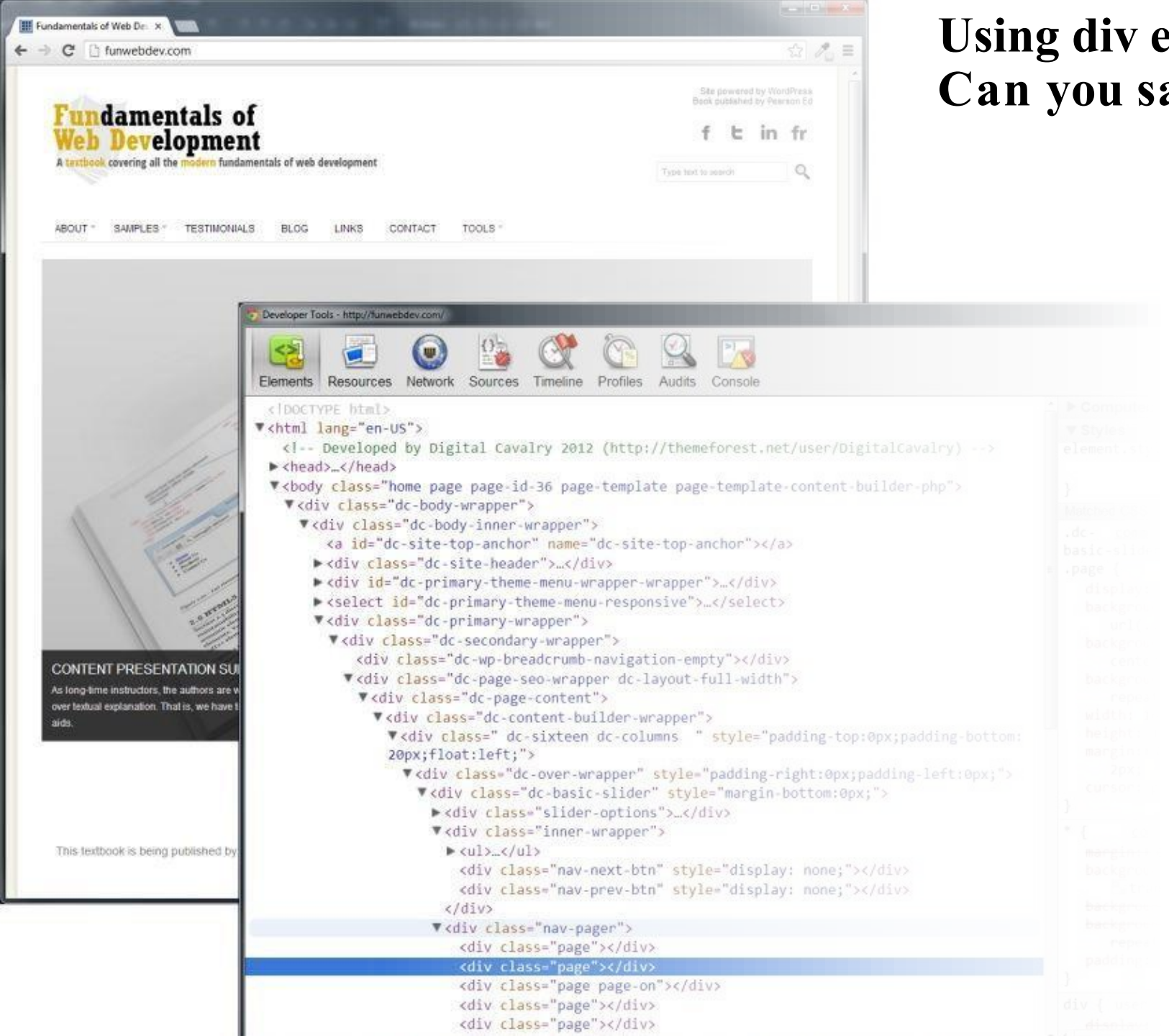
```
<!DOCTYPE html>
<html>
<body>
<p>goodmorning<br><del>Good evening</del><br> good night </p>
</body>
</html>
```

Divisions

- This ** ** tag is also a container element and is used to create a logical grouping of content
  - The element has no intrinsic presentation.
  - It is frequently used in contemporary CSS-based layouts to mark out sections.



Using div elements
Can you say“div-tastic”



HTML5 has a variety of new semantic elements (which we will examine later) that can be used to reduce somewhat the confusing mass of div within divs within divs that is so typical of contemporary web design.

Links <a>

- Links are created using the **<a>** element (the “a” stands for anchor).
- A link has two main parts: the destination and the label.

```
<a href="http://www.centralpark.com">Central Park</a>
```

Destination

Label (text)

```
<a href="index.html"><img src="logo.gif" /></a>
```

Label (image)

Types of Links

Links to external sites (or to individual resources such as images or movies on an external site).

Links to other pages or resources within the current site.

Links to other places within the current page.

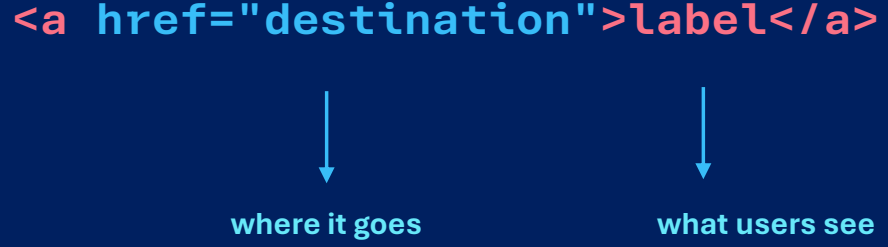
Links to particular locations on another page.

Links that are instructions to the browser to start the user's email program

Links that are instructions to the browser to execute a Javascript function.

Common href patterns

The href attribute points a link to a destination. These are the patterns shown in the source image.



Rule of thumb

Use absolute URLs for external sites, relative URLs for your own pages, and fragments (#id) for sections.

Examples from the markup

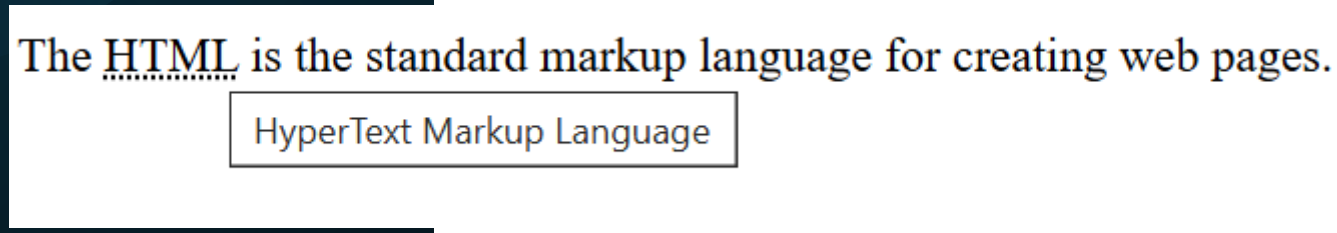
href value	does this
http://www.centralpark.com	External website
http://www.centralpark.com/logo.gif	Image file
index.html	Local page
#top	Jump on this page
productX.html#reviews	Section on another page
mailto://person@somewhere.com	Open email client
javascript://OpenAnnoyingPopup();	Run script action
tel:+18009220579	Tap-to-call number

```
<a href="productX.html#reviews">Read reviews</a>
```

HTML <abbr> Tag

The <abbr> tag defines an abbreviation or an acronym, like "HTML", "CSS", "Mr.", "Dr.", "ASAP", "ATM".

The <abbr title="HyperText Markup Language">HTML</abbr> is the standard markup language for creating web pages.



HTML <figure> Tag

The <figure> tag specifies self-contained content, like illustrations, diagrams, photos, code listings, etc.

While the content of the <figure> element is related to the main flow, its position is independent of the main flow, and if removed it should not affect the flow of the document.

Tip: The <figcaption> element is used to add a caption for the <figure> element.

```
<figure>
  <img src="https://via.placeholder.com/150" alt="Sample Image">
  <figcaption>This is a caption describing the image.</figcaption>
</figure>
```

Link Text

Some guidance … or … don't "Click Here"

Links with the label "Click Here" were once a staple of the web.

Today, such links are frowned upon, since:

- they do not tell users where the link will take them
- as a verb "click" is becoming increasingly inaccurate when one
- takes into account the growth of mobile browsers.

Instead, textual link labels should be descriptive.

"Click here to see the race results"

"Race Results" or **"See Race Results".**

```
<a href="race_results.html">Race Results</a>
```

When referencing a page or resource on an external site, a full
Absolute reference is required: that is

- the protocol (typically, http://),
- the domain name,
- any paths, and then finally
- the file name of the desired resource.



URL Absolute Referencing



For external resources

URL Relative Referencing An essential skill

- We also need to be able to successfully reference files within our site.
- This requires learning the syntax for so-called **relative referencing**.
- When referencing a resource that is on the same server as your HTML document, then you can use briefer relative referencing. If the URL does not include the “http://” then the browser will request the current server for the file.

URL Relative Referencing

- If all the resources for the site reside within the same **directory** (also referred to as a **folder**), then you can reference those other resources simply via their filename.
- However, most real-world sites contain too many files to put them all within a single directory.
- For these situations, a relative pathname is required along
- with the filename.
- The **pathname** tells the browser where to locate the file on the server.

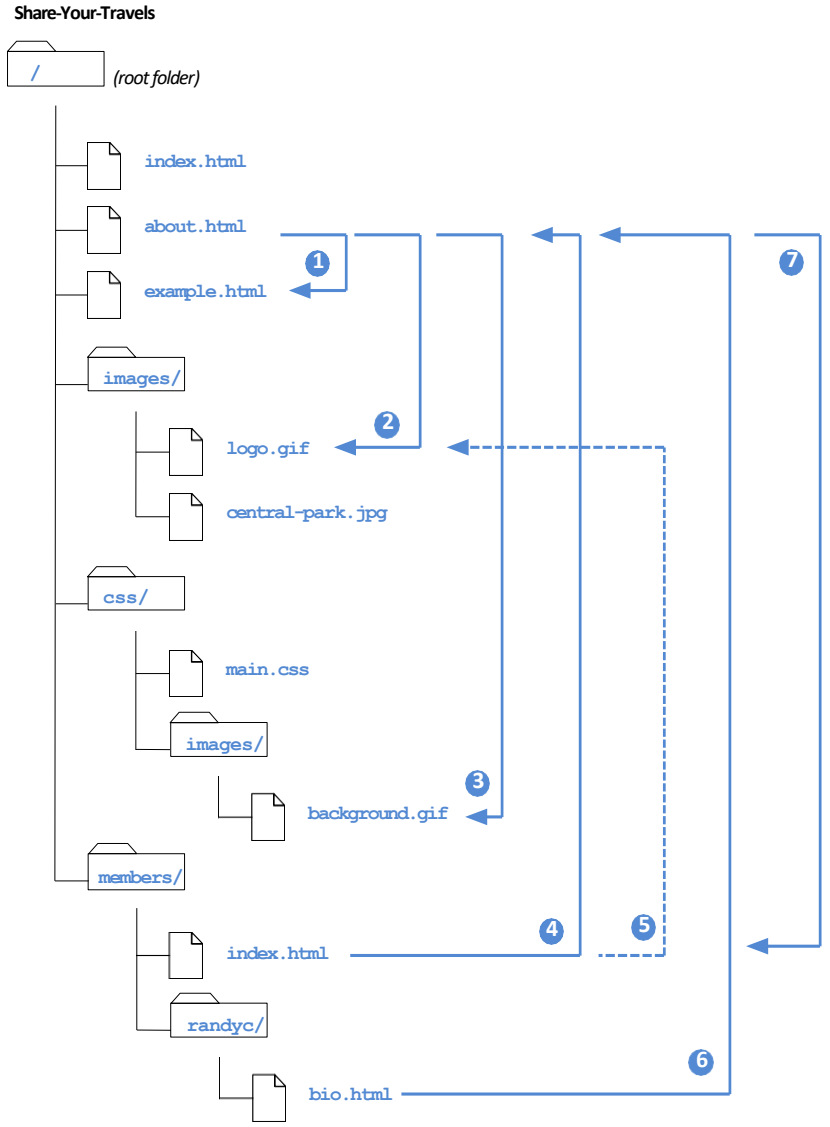
Pathnames

Pathnames on the web follow Unix conventions.

- Forward slashes (“/”) are used to separate directory names
- from each other and from file names.

- Double-periods (“..”) are used to reference a directory
- “above” the current one in the directory tree.

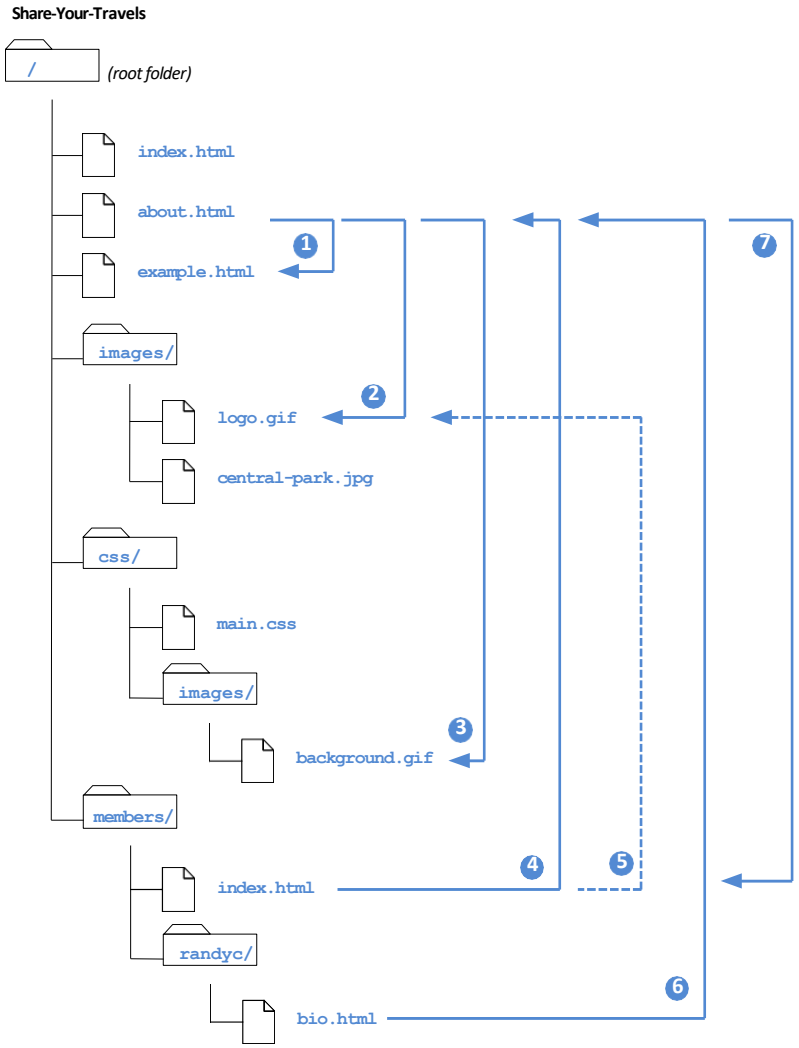
URL Relative Referencing



Relative Link Type	Example
1 **Same Directory** To link to a file within the same folder, simply use the file name.	To link to example.html from about.html (in Figure 2.18), use: `<a href="example.html">`
2 **Child Directory** To link to a file within a subdirectory, use the name of the subdirectory and a slash before the file name.	To link to logo.gif from about.html, use: `<a href="images/logo.gif">`
3 **Grandchild/Descendant Directory** To link to a file that is multiple subdirectories *below* the current one, construct the full path by including each subdirectory name (separated by slashes) before the file name.	To link to background.gif from about.html, use: `<a href="css/images/background.gif">`

4 **Parent/Ancestor Directory**
Use "../" to reference a folder *above* the current one. If trying to reference a file several levels above the current one, simply string together multiple "../".
	To link to about.html from index.html in members, use:
	`<a href="../about.html">`
	To link to about.html from bio.html, use:
	`<a href="../../about.html">`

URL Relative Referencing



5 Sibling Directory

Use "../"to move up to the appropriate level, and then use the same technique as for child or grandchild directories.

To link to about.html from index.html in members, use:

```
<a href="../images/about.html">
```

To link to background.gif from bio.html, use:

```
<a
href="../../css/images/background.
gif">
```

6 Root Reference

An alternative approach for ancestor and sibling references is to use the so-called **root reference** approach. In this approach, begin the reference with the root reference (the "/") and then use the same technique as for child or grandchild directories. **Note that these will only work on the server! That is, they will not work when you test it out on your local machine.**

To link to about.html from bio.html, use:

```
<a href="/about.html">
```

To link to background.gif from bio.html, use:

```
<a
href="/css/images/background.gif">
```

7 Default Document

Web servers allow references to directory names without file names. In such a case, the web server will serve the default document, which is usually a file called index.html (apache) or default.html (IIS). **Again, this will only work on the web server**

To link to index.html in members from about.html, use either:

```
<a href="members">
```

Or

```
<a href="/members">
```

Inline Text Elements



Inline elements do not disrupt the flow of text (i.e., cause a line break).



HTML5 defines over 30 of these elements.

Inline elements in HTML

Element	Description	Example
<a>	Anchor (hyperlink)	Link
<abbr>	Abbreviation	<abbr title="HyperText Markup Language">HTML</abbr>
<acronym>	Acronym (obsolete, use <abbr>)	<acronym title="World Wide Web">WWW</acronym>
	Bold text	Bold Text
<bdo>	Bi-directional override	<bdo dir="rtl">Right to left</bdo>
<big>	Larger text (obsolete)	<big>Big Text</big>
 	Line break	Line one Line two
<button>	Button element	<button>Click me</button>
<cite>	Citation	<cite>Shakespeare, Hamlet</cite>
<code>	Code snippet	<code>console.log('Hello');</code>
<dfn>	Definition term	<dfn>HTML</dfn>
	Emphasis	Important
	Italic text	Italic Text
<img>	Image	<img src="img.png" alt="Sample Image">
<input>	Input field	<input type="text" name="name">
<kbd>	Keyboard input	<kbd>Ctrl + C</kbd>

Inline elements in HTML

Element	Description	Example
<label>	Label for form control	<label for="name">Name:</label>
<map>	Image map	<map name="map1">...</map>
<object>	Embeds multimedia	<object data="movie.swf"></object>
<output>	Output result	<output name="result">42</output>
<q>	Inline quote	<q>Quote text</q>
<samp>	Sample output	<samp>Error message</samp>
<script>	JavaScript code	<script>console.log('Hi');</script>
<select>	Dropdown list	<select><option>One</option></select>
<small>	Fine print text	© 2025
	Generic inline container	Red text
	Important text	Warning!
<sub>	Subscript text	H₂O
<sup>	Superscript text	x²
<textarea>	Multiline text input	<textarea name="comments"></textarea>
<time>	Time or date	<time datetime="2025-07-16">July 16, 2025</time>
<tt>	Teletype or monospaced text (obsolete)	<tt>Monospaced</tt>
<var>	Variable	<var>x</var>



Lists

HTML provides three types of lists

- **Unordered lists**. Collections of items in no particular order; these are by default rendered by the browser as a bulleted list.
- **Ordered lists**. Collections of items that have a set order; these are by default rendered by the browser as a numbered list.
- **Definition lists**. Collection of name and definition pairs. These tend to be used infrequently. Perhaps the most common example would be a FAQ list.

Unordered HTML List

- An unordered list starts with the <ul> tag.
- Each list item starts with the <li> tag.

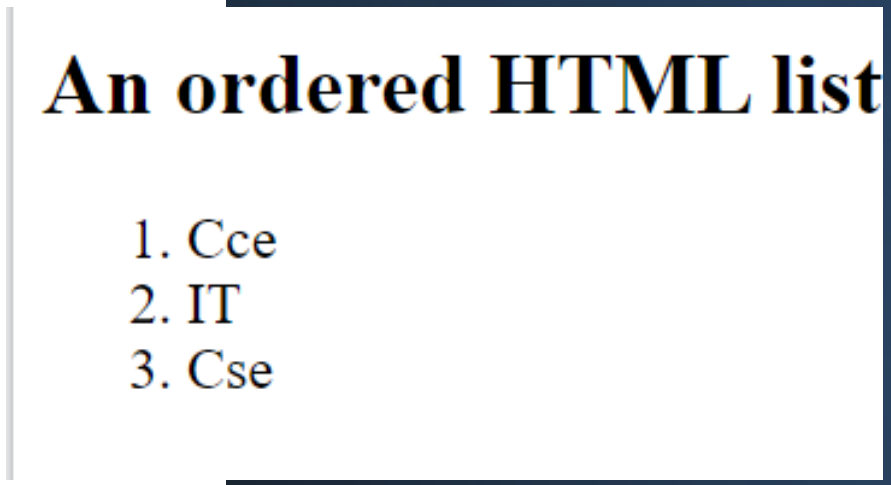
```
<!DOCTYPE html>
<html>
<body>
<h2>An unordered HTML list</h2>
<ul>
<li>CCE</li>
<li>IT</li>
<li>CSE</li>
</ul>
</body>
</html>
```



Ordered HTML List

- An ordered list starts with the <ol> tag.
- Each list item starts with the <li> tag.

```
<!DOCTYPE html>
<html>
<body>
<h2>An unordered HTML list</h2>
<ol>
<li>CCE</li>
<li>IT</li>
<li>CSE</li>
</ol>
<body>
</html>
```





Description HTML List

- An description list starts with the <dl> tag.
- A description list is a list of terms, with a description of each term.
- Use the HTML <dl> element to define a description list
- Use the HTML <dt> element to define the description term
- Use the HTML <dd> element to describe the term in a description
- list

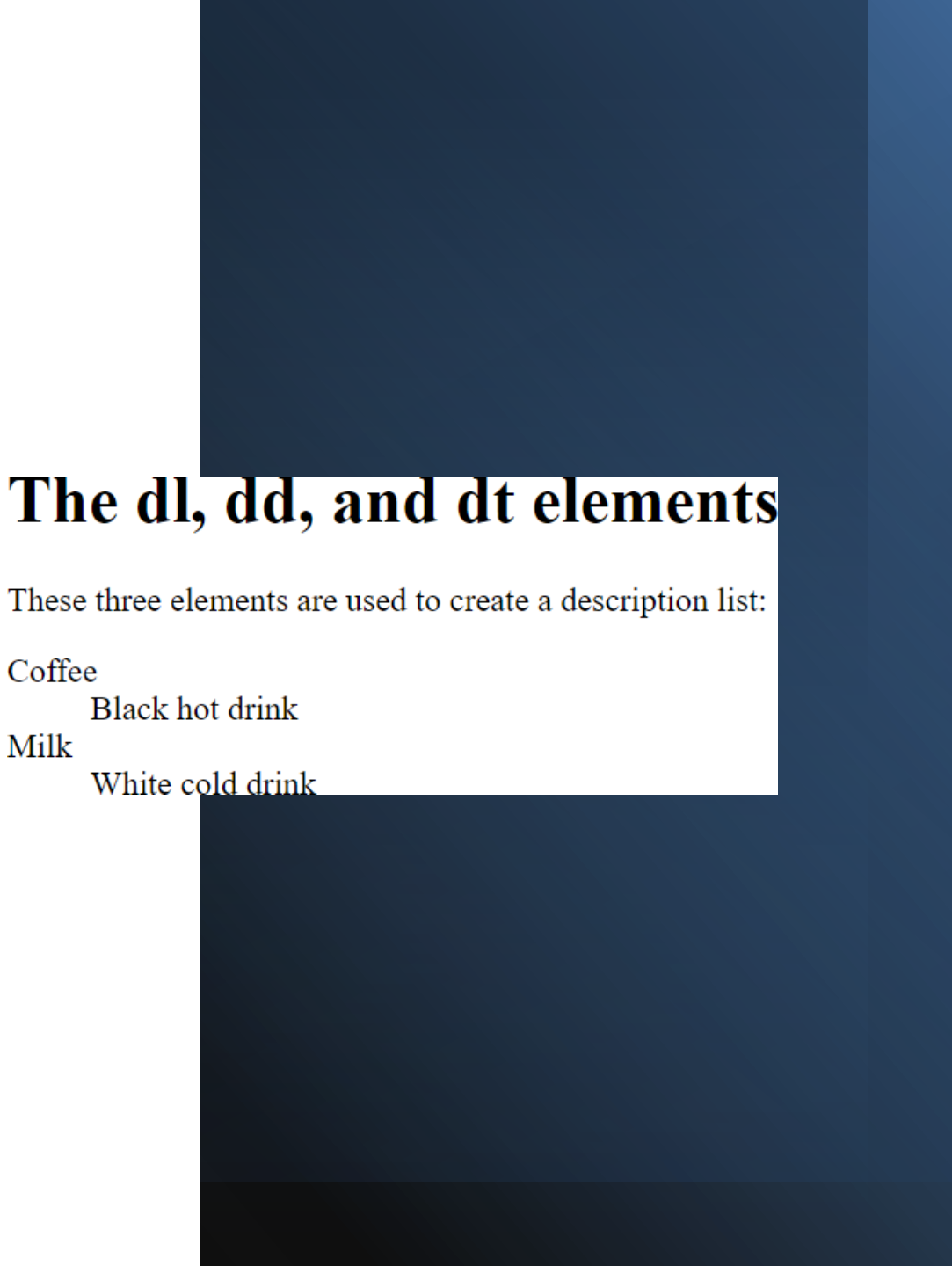
```
<!DOCTYPE html>
<html>
<body>

<h1>The dl, dd, and dt elements</h1>

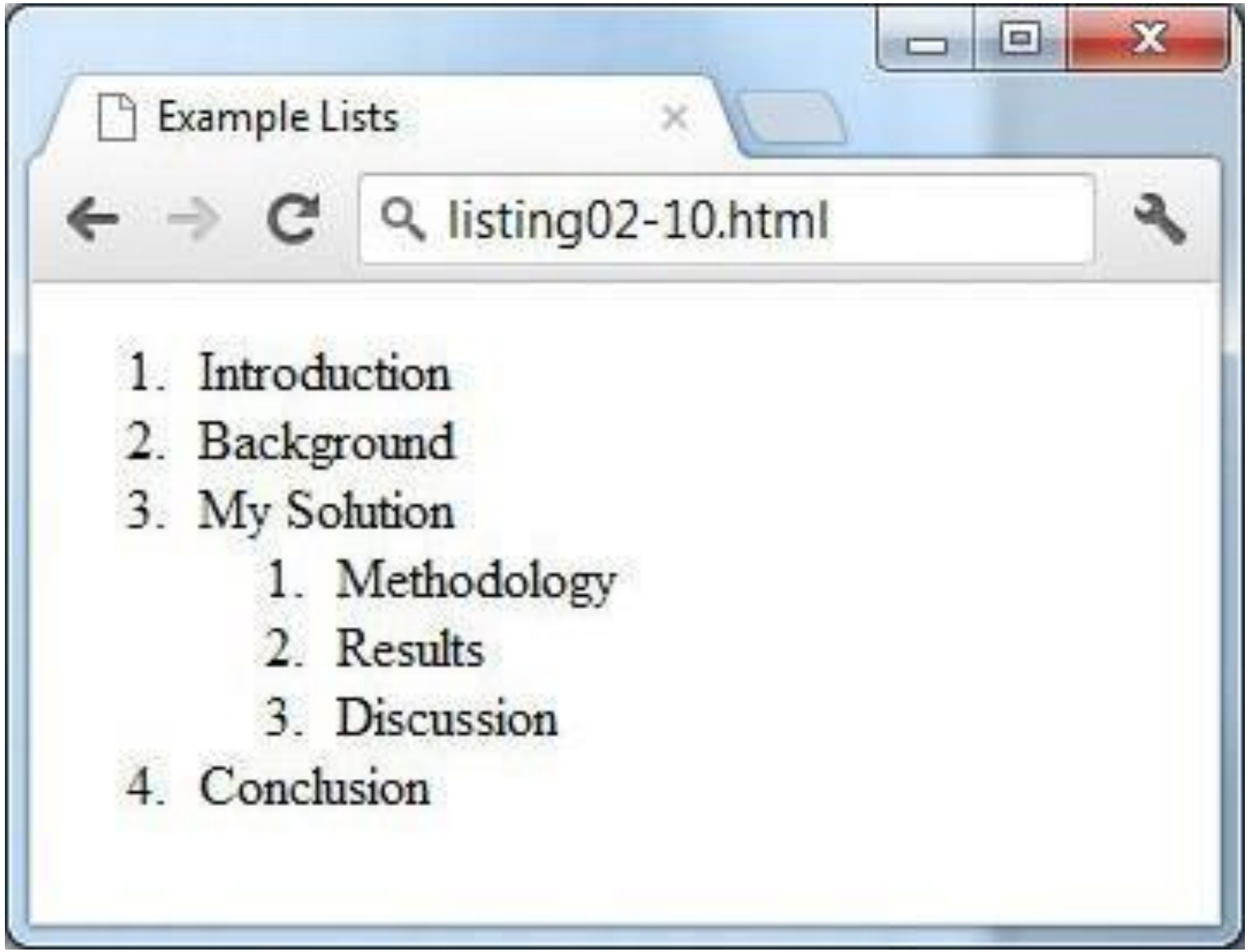
<p>These three elements are used to
create a description list:</p>

<dl>
<dt>Coffee</dt>
<dd>Black hot drink</dd>
<dt>Milk</dt>
<dd>White cold drink</dd>
</dl>

</body>
</html>
```



Write the HTML code



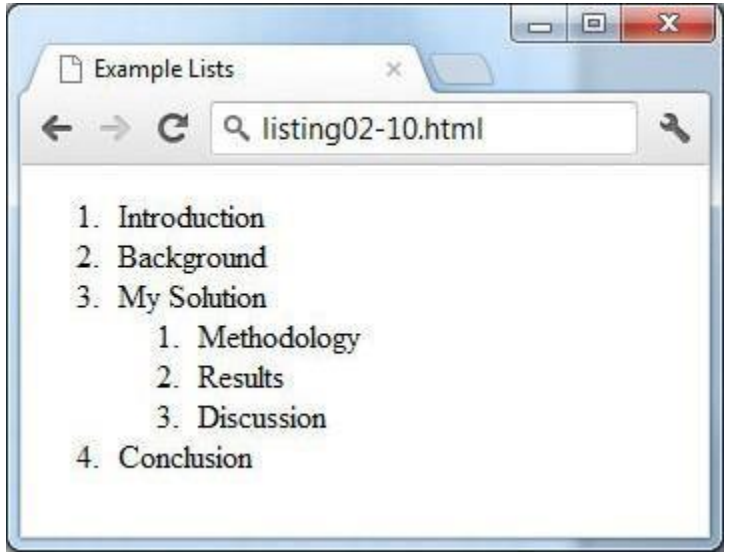
Lists

Notice that the list item element can contain other HTML elements

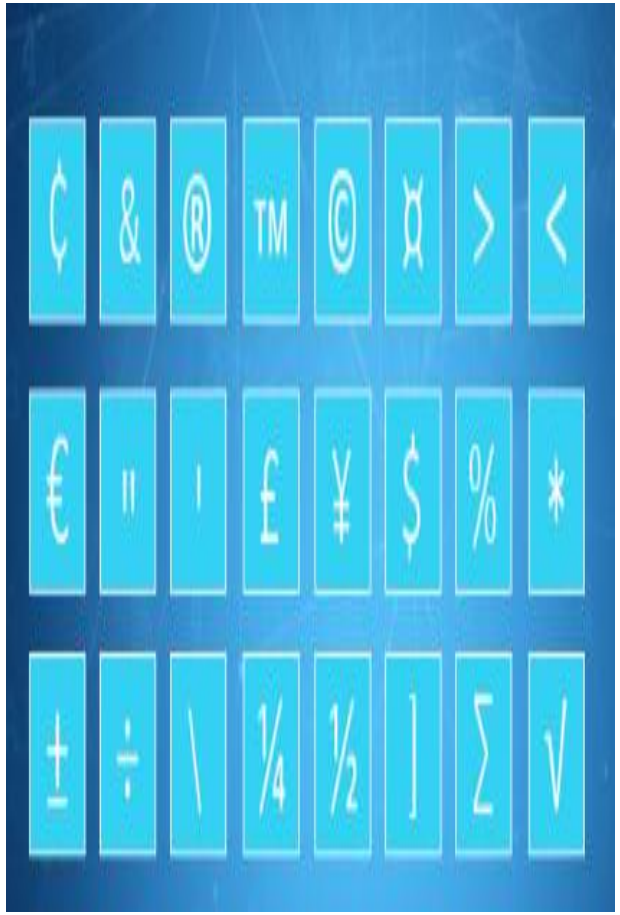
```
<ul>
    <li><a href="index.html">Home</a></li>
    <li>About Us</li>
    <li>Products</li>
    <li>Contact Us</li>
</ul>
```

```
<ol >
    <li>Introduction</li>
    <li>Background</li>
    <li>My Solution</li>
    <li>
      <ol>
        <li>Methodology </li>
        <li>Results</li>
        <li>Discussion</li>
      </ol>
    </li>
    <li>Conclusion</li>
</ol>
```





Character Entities



- These are special characters for symbols for which there is either no way easy way to type in via a keyboard (such as the copyright symbol or accented characters) or which have a reserved meaning in HTML (for instance the “<” or “>” symbols).
- Entity names look like this:&entity_name;
- Entity numbers look like this:&#entity_number;
- They can be used in an HTML document by using the entity name or the entity number.

e.g.,   and ©

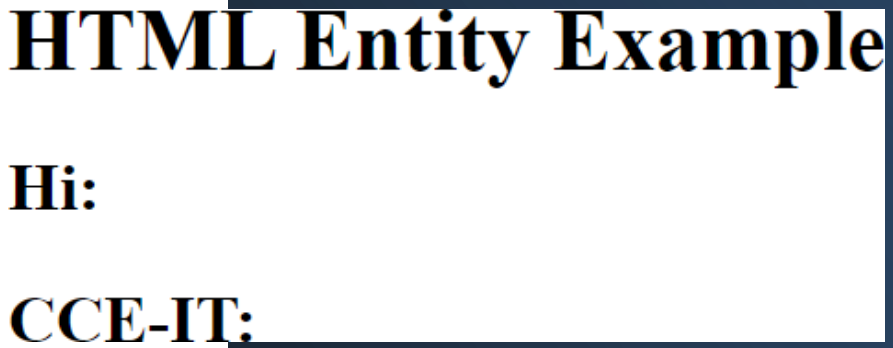
non-breaking space

```
<!DOCTYPE html>
<html>
<body>

<h1>HTML Entity Example</h1>

<h2>          Hi: </h2>
<h2>CCE-IT:</h2>

</body>
</html>
```



Common character entities

Entity Name	Entity Number	Description
		Nonbreakable space. The browser ignores multiple spaces in the source HTML file. If you need to display multiple spaces, you can do so using the nonbreakable space entity.
<	<	Less than symbol ("<").
>	>	Greater than symbol (">").
©	©	The © copyright symbol.
€	€	The € euro symbol.
™	™	The ™ trademark symbol.
ü	ü	The ü — i.e., small u with umlaut mark.

SEMANTIC MARKUP

Element	Description	Example
<a>	Anchor used for hyperlinks.	Visit Example
<abbr>	An abbreviation	<abbr title="HyperText Markup Language">HTML</abbr>
** **	Line break	First line Second line
<cite>	Citation (i.e., a reference to another work).	<cite>Shakespeare, William. Hamlet.</cite>
<code>	Used for displaying code, such as markup or programming code.	<code>console.log('Hello');</code>
****	Emphasis	This text is emphasized

Element	Description	Example
<mark>	For displaying highlighted text	This is <mark>highlighted</mark> text.
<small>	For displaying the fine-print, i.e., "non-vital" text, such as copyright or legal notices.	© 2025 My Company
	The inline equivalent of the element. It is generally used to mark text that will receive special formatting using CSS.	Red text
	For content that is strongly important.	Important!
<time>	For displaying time and date data	<time datetime="2025-07-16">July 16, 2025</time>



Semantic Markup What does it mean?

- Over the past decade, a strong and broad consensus has grown around the belief that HTML documents should **only** focus on the structure of the document.
- Information about how the content should look when it is displayed in the browser is best left to CSS (Cascading Style Sheets).
- As a consequence, beginning HTML authors are often counseled to create **semantic HTML** documents.
- That is, an HTML document should not describe how to visually present content, but only describe its content's structural semantics or meaning.

Semantic Markup
Its advantages

- Eliminating presentation-oriented markup and writing semantic HTML
- Markup has a variety of important advantages:

  - **Maintainability**. Semantic markup is easier to update and change than web pages that contain a great deal of presentation markup.

  - **Faster**. Semantic web pages are typically quicker to author and
  - faster to download.

  - **Accessibility**. Visiting a web page using screen reading software can be a very frustrating experience if the site does not use semantic markup.

  - **Search engine optimization**. Semantic markup provides better instructions for search engines: it tells them what things are important content on the site.



Header and Footer
<header> <footer>

Most web site pages have a recognizable header and footer section.

Typically the **header** contains

- the site logo
- title (and perhaps additional subtitles or taglines)
- horizontal navigation links, and
- perhaps one or two horizontal banners.

Header and Footer
<header> <footer>

The typical footer contains less important material, such as

- smaller text versions of the navigation,
- copyright notices,
- information about the site's privacy policy, and
- perhaps twitter feeds or links to other social sites.

Header and Footer

- Both the HTML5 <header> and <footer> element can be used not only for *page* headers and footers, they can also be used for header and footer elements within other HTML5 containers, such as <article> or <section>.

```
<header>
    <img src="logo.gif" alt="logo" />
    <h1>Fundamentals of Web Development</h1>
    ...
</header>
<article>
    <header>
        <h2>HTML5 Semantic Structure Elements
</h2>
        <p>By <em>Randy Connolly</em></p>
        <p><time>September 30, 2012</time></p>
    </header>
    ...
</article>
```

Heading Groups <hgroup>

The <hgroup> element can be used to group related headings together within one container.

```
<header>
<hgroup>
  <h1>Chapter Two: HTML 1</h1>
  <h2>An Introduction</h2>
</hgroup>
</header>
<article>
<hgroup>
  <h2>HTML5 Semantic Structure Elements
  </h2>
  <h3>Overview</h3>
</hgroup>
</article>
```

Navigation <nav>

❖ The **<nav>** element represents a section of a page that contains links to other pages or to other parts within the same page.

❖Like the other new HTML5 semantic elements, the browser does not apply any special presentation to the <nav> element.

❖ The <nav> element was intended to be used for major navigation blocks, presumably the global and secondary navigation systems.

Navigation

```
<header>
  <img src="logo.gif"        alt="logo"    />
  <h1>Fundamentals of    Web Development</h1>
  <nav role="navigation">
     <ul>
        <li><a  href="index.html">Home</a></li>
        <li><a href="about.html">About Us</a></li>
        <li><a  href="browse.html">Browse</a></li>
     </ul>
  </nav>
</header>
```

Articles and Sections

<article> <section>

- The **<article>** element represents a section of content that forms an independent part of a document or site; for example, a magazine or newspaper article, or a blog entry.
- The **<section>** element represents a section of a document, typically with a title or heading.
- According to the W3C, **<section>** is a much broader element, while the **<article>** element is to be used for blocks of content that could potentially be read or consumed independently of the other content on the page.

```
<section>
<h2>Latest News</h2>
<article>
 <h3>New Research on Climate
 Change</h3>
 <p>A recent study shows...</p>
</article>
<article>
 <h3>Tech Innovations in 2024</h3>
 <p>Tech companies are focusing
 on...</p>
</article>
</section>
```



Sections versus Divs
How to decide which to use

- The WHATWG specification warns readers that the

<section> element is **not** a generic container element. HTML already has the element for such uses.

- When an element is needed only for styling purposes or as a convenience for scripting, it makes sense to use the element instead.

- Another way to help you decide whether or not to use the
- <section> element is to ask yourself if it is appropriate for the element's contents to be listed explicitly in the document's outline.

- If so, then use a <section>; otherwise use a .



Figure and Figure Captions

<figure> <figcaption>

The W3C Recommendation indicates that the <figure> element can be used not just for images but for any type of *essential* content that could be moved to a different location in the page or document and the rest of the document would still make sense.

The **<figure>** element should **not** be used to wrap every image.

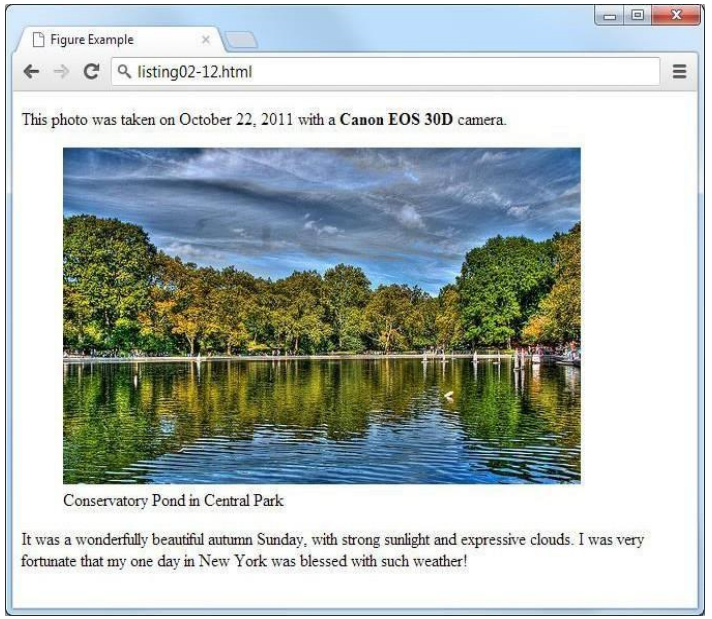
For instance, it makes no sense to wrap the site logo or non- essential images such as banner ads and graphical embellishments within <figure> elements.

Instead, only use the <figure> element for circumstances where the image (or other content) has a caption and where the figure is essential to the content but its position on the page is relatively unimportant.

Figure and Figure Captions

Figure could be moved to a different location in document …
But it has to exist in the document (i.e., the figure isn't optional)

```
<p>This photo was taken on October 22, 2011 with a Canon EOS30D camera.</p>
<figure>
  <img src="images/central-park.jpg" alt="Central Park" /><br/>
  <figcaption>Conservatory Pond in Central Park</figcaption>
</figure>
<p>
It was a wonderfully beautiful autumn Sunday, with strong sunlight and
expressive clouds. I was very fortunate that my one day in NewYork was
blessed with such weather!
</p>
```



Aside <aside>

- The **<aside>** element is similar to the **<figure>** element in that it is used for marking up content that is separate from the main content on the page.

- But while the **<figure>** element was used to indicate important information whose location on the page is somewhat unimportant, the **<aside>** element "represents a section of a page that consists of content that is tangentially related to the content around the aside element."

- The **<aside>** element could thus be used for sidebars, pull quotes, groups of advertising images, or any other grouping of non-essential elements.

HTML <meta> Tag

- The <meta> tag defines metadata about an HTML document. Metadata is data (information) about data.

- <meta> tags always go inside the <head> element, and are typically used to specify character set, page description, keywords, author of the document, and viewport settings.

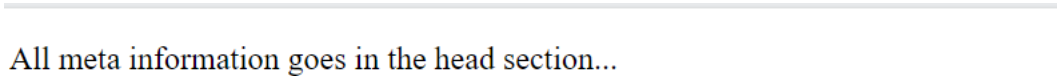
- Metadata will not be displayed on the page, but is machine
- parsable.

- Metadata is used by browsers (how to display content or reload page), search engines (keywords), and other web services.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <meta name="description" content="Free Web tutorials">
  <meta name="keywords" content="HTML,CSS,XML,JavaScript">
  <meta name="author" content="John Doe">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
</head>
<body>

<p>All meta information goes in the head section...</p>

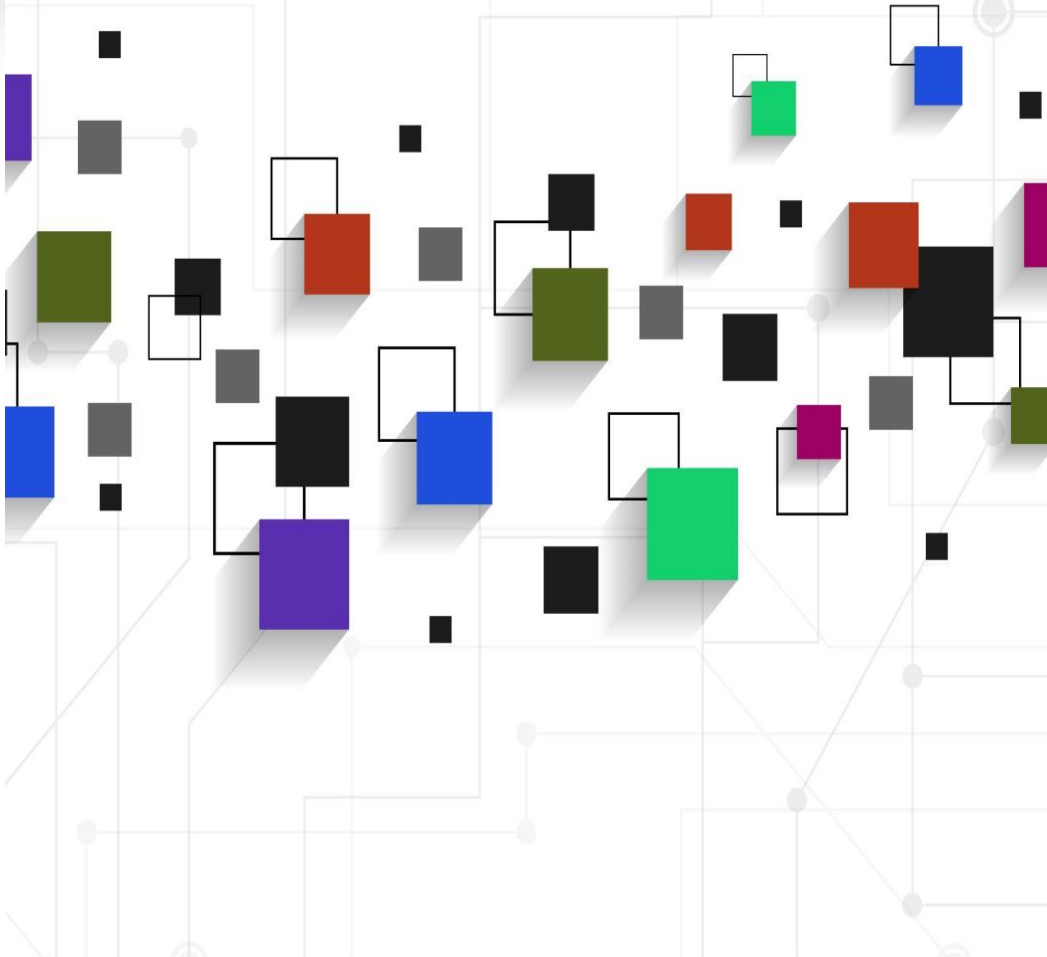
</body>
</html>
```

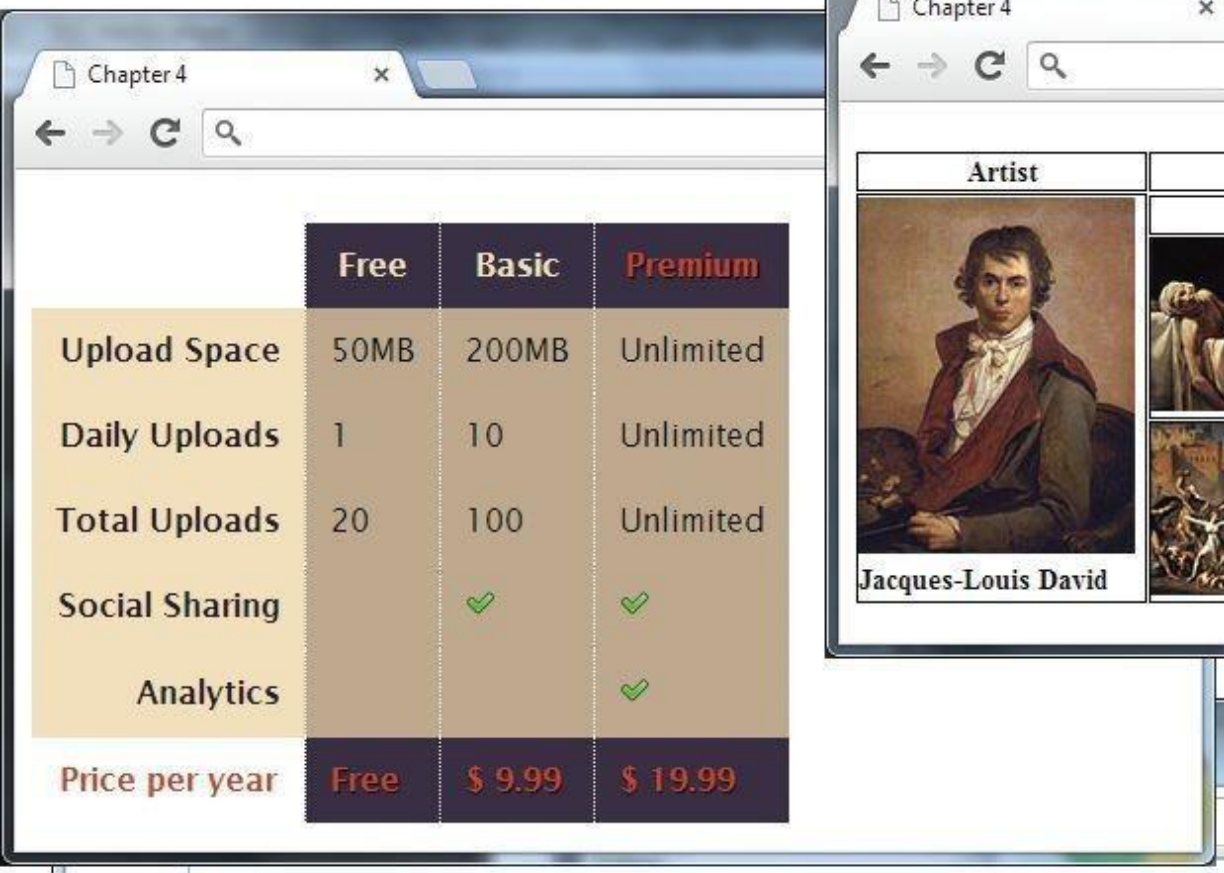


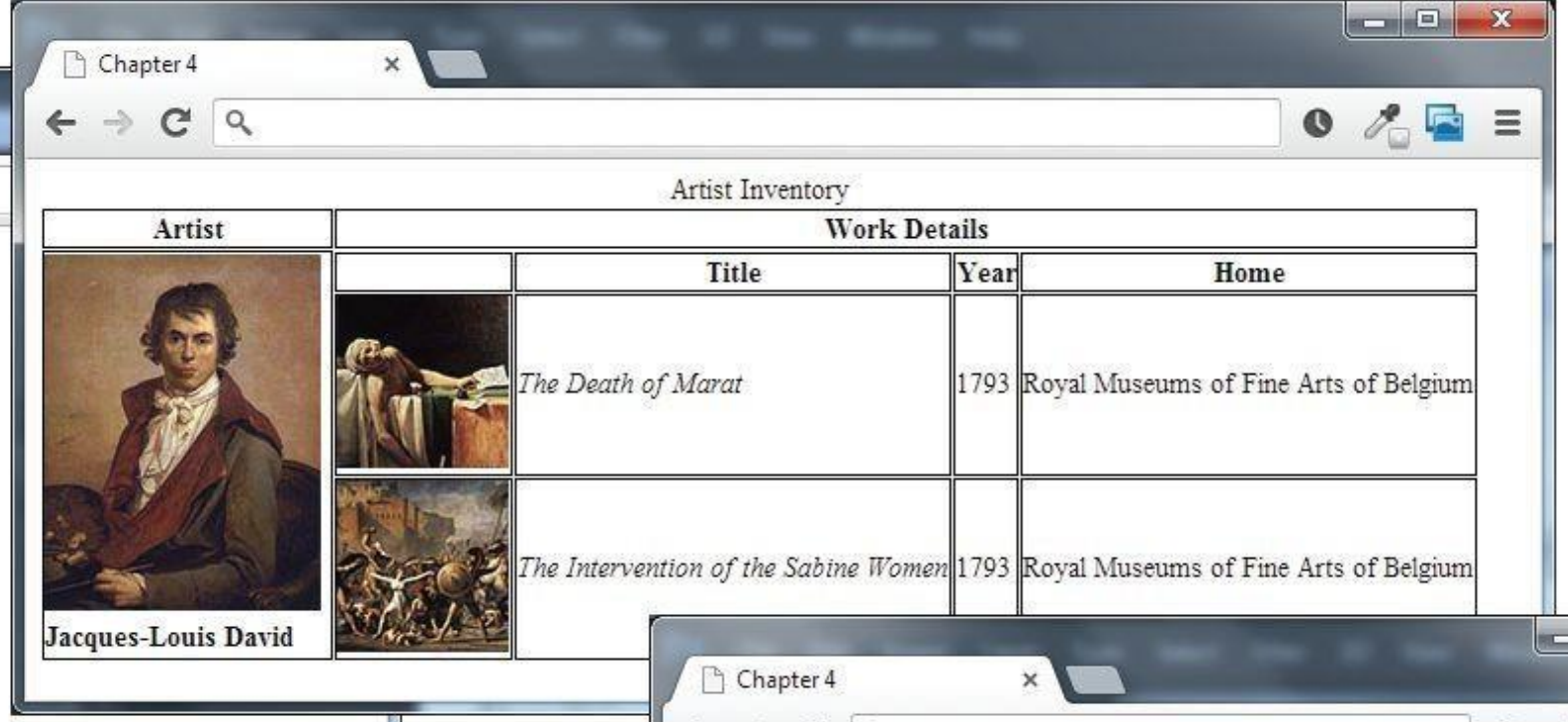
HTML Tables and Forms

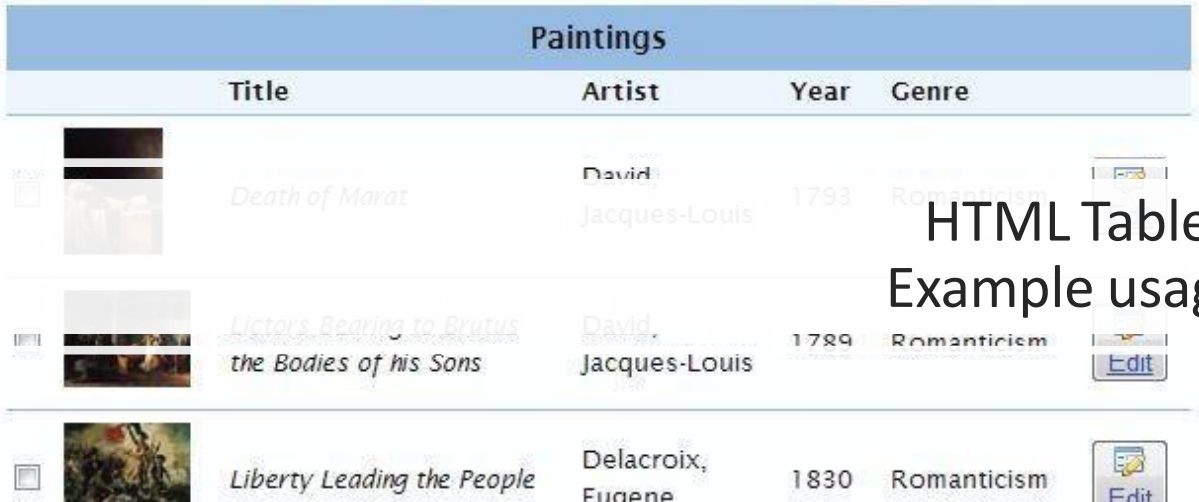
HTML Tables
A grid of cells

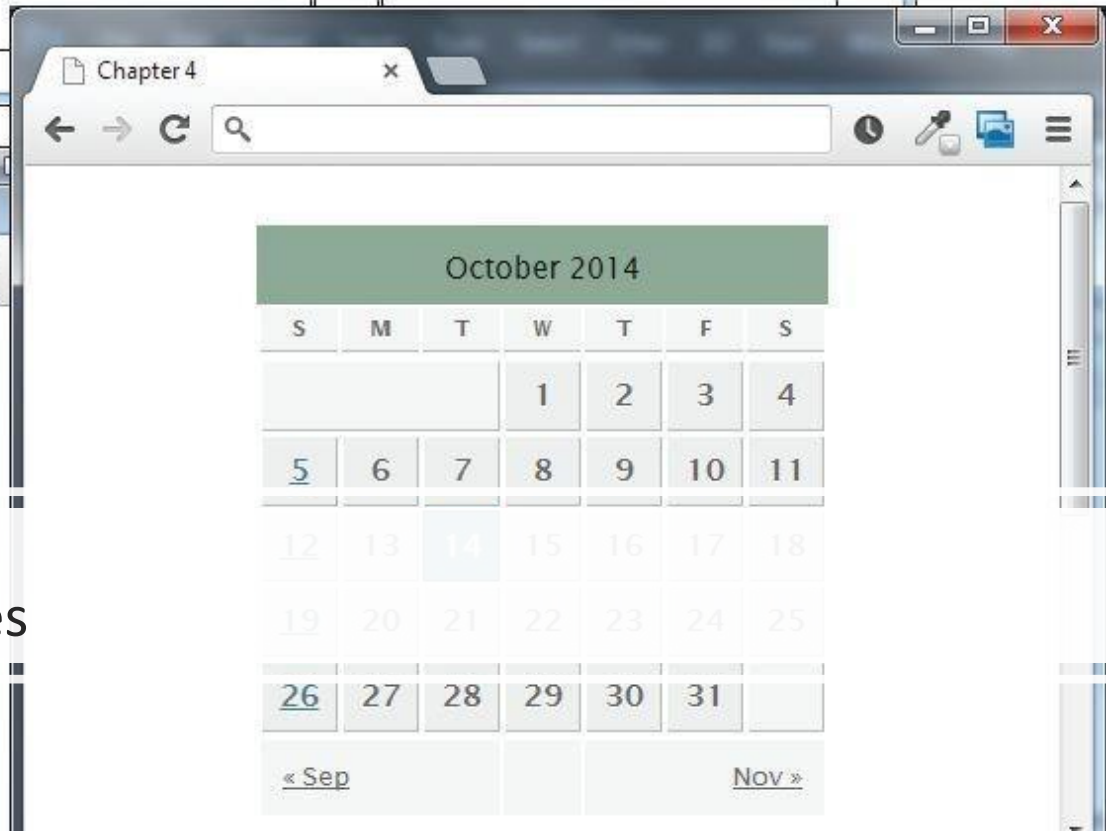
- A table in HTML is created using the **<table>** element Tables can be used to display:
- Many types of content
  - Calendars, financial data, lists, etc…
- Any type of data
  - Images
  - Text
  - Links
  - Other tables











HTML Tables
Example usages

Tables Basics

Rows and cells

- An HTML **<table>** contains any number of rows (**<tr>**)
- Each row contains any number of table data cells (**<td>**)
- Content goes inside of **<td></td>** tags

```
<table>
    <tr>
        <td>The Death of Marat</td>
    </tr>
</table>
```



content

A basic Example

<table>
<tr>
<td>The Death of Marat</td>
<td>Jacques-Louis David</td>
<td>1793</td>
<td>162cm</td>
<td>128cm</td>
</tr>
<tr>
<td>Burial at Ornans</td>
<td>Gustave Courbet</td>
<td>1849</td>
<td>314cm</td>
<td>663cm</td>
</tr>
</table>

```
<table>
    <tr>
        <td>The Death of Marat</td>
        <td>Jacques-Louis David</td>
        <td>1793</td>
        <td>162cm</td>
        <td>128cm</td>
    </tr>
    <tr>
        <td>Burial at Ornans</td>
        <td>Gustave Courbet</td>
        <td>1849</td>
        <td>314cm</td>
        <td>663cm</td>
    </tr>
</table>
```



With Table Headings

<table>
<tr><td><tr></td><td>Title <th></td><td>Artist <th></td><td>Year <th></td><td>Width <th></td><td>Height <th></td></tr>
<tr><td><tr></td><td>The Death of Marat <td></td><td>Jacques-Louis David <td></td><td>1793 <td></td><td>162cm <td></td><td>128cm <td></td></tr>
<tr><td><tr></td><td>Burial at Ornans <td></td><td>Gustave Courbet <td></td><td>1849 <td></td><td>314cm <td></td><td>663cm <td></td></tr>
</table>

```
<table>
   <tr>
      <th>Title</th>
      <th>Artist</th>
      <th>Year</th>
      <th>Width</th>
      <th>Height</th>
   </tr>
   <tr>
      <td>The Death of Marat</td>
      <td>Jacques-Louis David</td>
      <td>1793</td>
      <td>162cm</td>
      <td>128cm</td>
   </tr>
   <tr>
      <td>Burial at Ornans</td>
      <td>Gustave Courbet</td>
      <td>1849</td>
      <td>314cm</td>
      <td>663cm</td>
   </tr>
</table>
```

th





Importance of Table Headings

- A table heading <th>
  - Browsers tend to make the content within a <th> element bold
  - <th> element for accessibility (it helps those using screen readers)
  - Provides some semantic info about the row being a row of headers

Spanning Rows and Columns

Span Span Span a Row

Each row must have the same number of <td> or <th> containers. If you want a given cell to cover several columns or rows,

<table>

<tr>	Title <th>	Artist <th>	Year <th>	Size (width x height) <th colspan=2>	
<tr>	The Death of Marat <td>	Jacques-Louis David <td>	1793 <td>	162cm <td>	128cm <td>
<tr>	Burial at Ornans <td>	Gustave Courbet <td>	1849 <td>	314cm <td>	663cm <td>

use the **colspan or rowspan attributes**

Notice that this row now only has four cell elements.

```
<table>
   <tr>
      <th>Title</th>
      <th>Artist</th>
      <th>Year</th>
      <th colspan="2">Size (width x height)</th>
   </tr>
   <tr>
      <td>The Death of Marat</td>
      <td>Jacques-Louis David</td>
      <td>1793</td>
      <td>162cm</td>
      <td>128cm</td>
   </tr>
   ...
</table>
```

Using Tables for Layout

Popular in the 1990s:
Tables were widely used to control page layout because CSS was not mature or widely supported at the time.

Works in many situations:
Tables could create complex page designs that worked consistently across browsers.

Results in table bloat:
Using tables for layout often leads to large, complex HTML code with many nested tables, making pages harder to maintain.

Not semantic:
Tables were designed for tabular data, not for page layout. Using them for layout mixes content with presentation, which is against modern web standards and accessibility best practices.

Larger HTML pages:
Table layouts increase HTML file size, impacting page load speed.

Browser quirks:
Different browsers may render table-based layouts inconsistently, causing cross-browser issues.

Using Tables for Layout Contd..

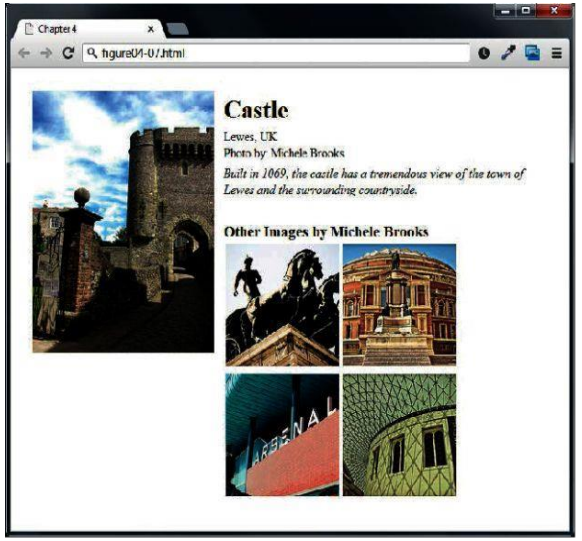
<table>

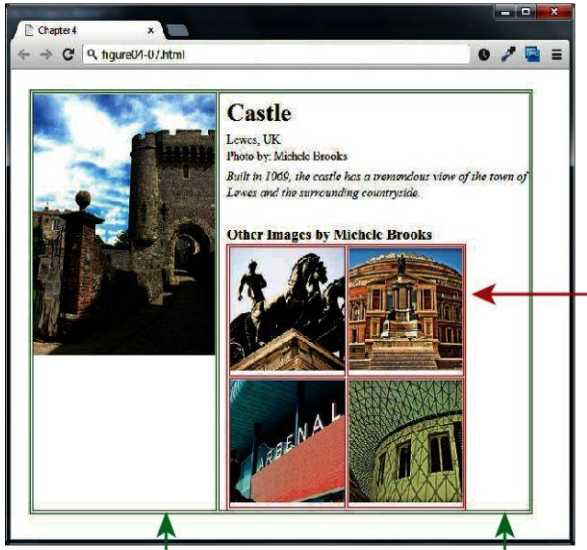
Artist <th>	Title <th>	Year <th>	<tr>
Jacques-Louis David <td rowspan=3>	The Death of Marat <td>	1793 <td>	<tr>
	The Intervention of the Sabine Women <td>	1799 <td>	<tr>
	Napoleon Crossing the Alps <td>	1800 <td>	<tr>

```
<table>
   <tr>
      <th>Artist</th>
      <th>Title</th>
      <th>Year</th>
   </tr>
   <tr>
      <td rowspan="3">Jacques-Louis David</td>
      <td>The Death of Marat</td>
      <td>1793</td>
   </tr>
   <tr>
      <td>The Intervention of the Sabine Women</td>
      <td>1799</td>
   </tr>
   <tr>
      <td>Napoleon Crossing the Alps</td>
      <td>1800</td>
   </tr>
   ...
</table>
```

Notice that these two rows now only have two cell elements.

Example
Table layouts






```
<table>
  <tr>
    <td>
      <img src="images/959.jpg" alt="Castle"/>
    </td>
    <td>
      <h2>Castle</h2>
      <p>Lewes, UK</p>
      <p>Photo by: Michele Brooks</p>
      <p>Built in 1069, the castle has a tremendous
         view of the town of Lewes and the
         surrounding countryside.
      </p>

      <h3>Other Images by Michele Brooks</h3>

      <table>
        <tr>
          <td><img src="images/464.jpg" /></td>
          <td><img src="images/537.jpg" /></td>
        </tr>
        <tr>
          <td><img src="images/700.jpg" /></td>
          <td><img src="images/828.jpg" /></td>
        </tr>
      </table>
    </td>
  </tr>
</table>
```

Additional table tags

- <caption>
- <col>,<colgroup>
- <thead>
- <tfoot>
- <tbody>

A title for the table is good for accessibility.

These describe our columns, and can be used to aid in styling.

Table header could potentially also include other <tr> elements.

Yes, the table footer comes *before* the body.

Potentially, with styling the browser can scroll this information, while keeping the header and footer fixed in place.

```
<table>
   <caption>19th Century French Paintings</caption>
   <col class="artistName" />
   <colgroup id="paintingColumns">
      <col />
      <col />
   </colgroup>

   <thead>
      <tr>
         <th>Title</th>
         <th>Artist</th>
         <th>Year</th>
      </tr>
   </thead>

   <tfoot>
      <tr>
         <td colspan="2">Total Number of Paintings</td>
         <td>2</td>
      </tr>
   </tfoot>

   <tbody>
      <tr>
         <td>The Death of Marat</td>
         <td>Jacques-Louis David</td>
         <td>1793</td>
      </tr>
      <tr>
         <td>Burial at Ornans</td>
         <td>Gustave Courbet</td>
         <td>1849</td>
      </tr>
   </tbody>

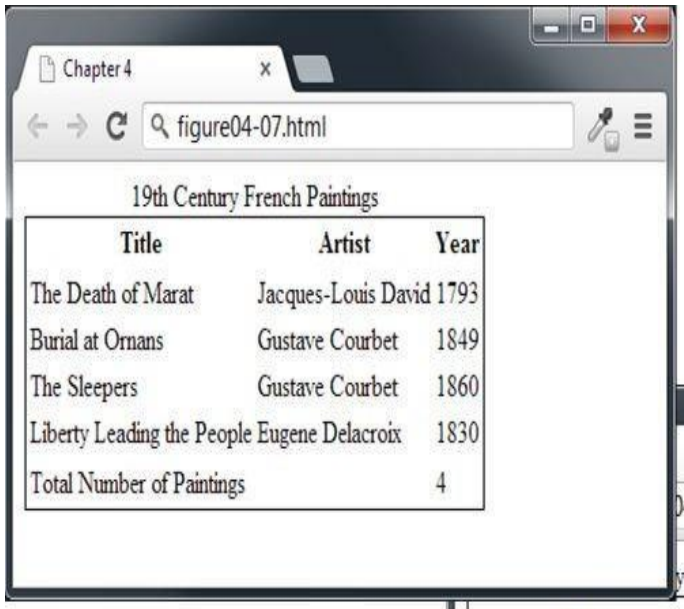
</table>
```



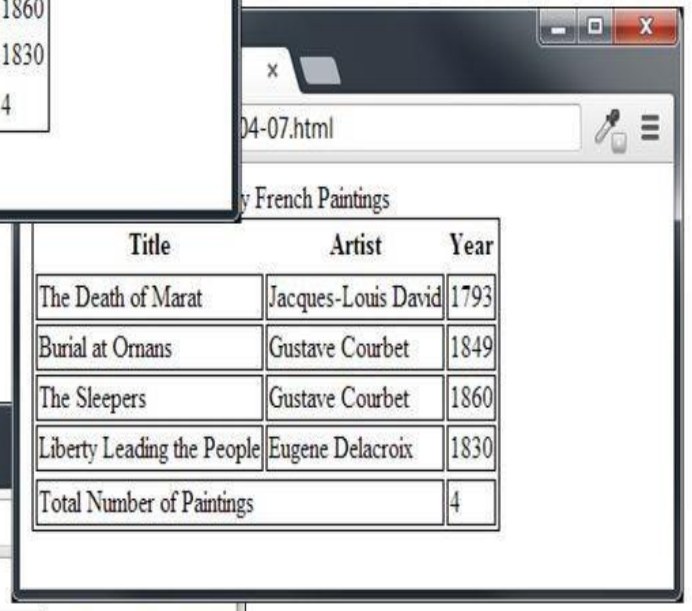
Styling Tables

- In HTML5 it is left to CSS, However legacy support for deprecated
- HTML attributes still exist
  - **width, height**—for setting the width and height of cells
  - **cellspacing**—for adding space between every cell in the table
  - **cellpadding**—for adding space between the content of the cell and its border
  - **bgcolor**—for changing the background color of any table element
  - **background**—for adding a background image to any table element
  - **align**—for indicating the alignment of a table in relation to the surrounding container

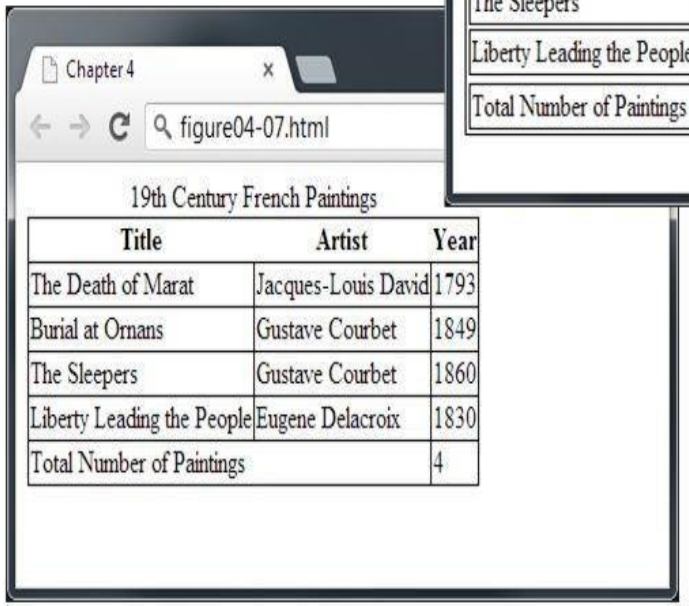
Styling Tables Borders




```
table {
   border: solid 1pt black;
}
```

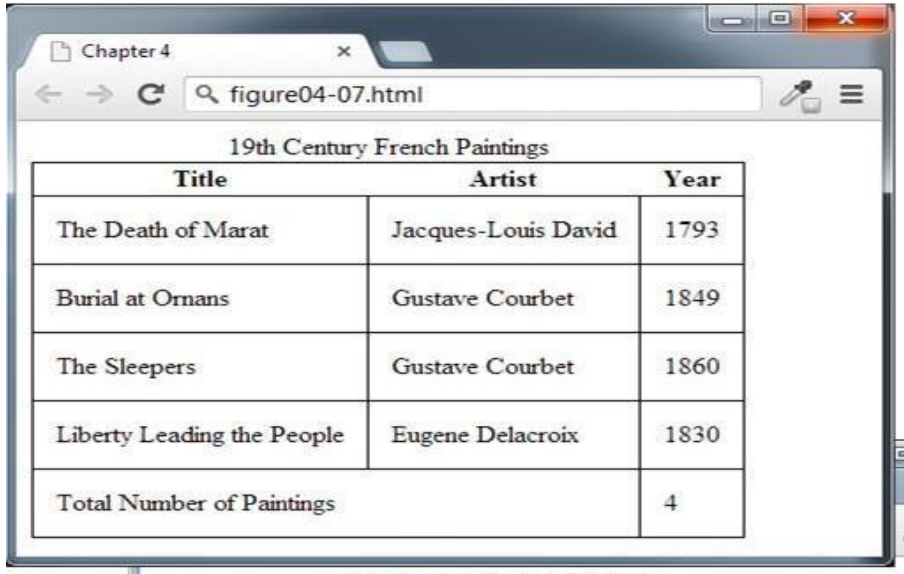



```
table {
   border: solid 1pt black;
}
td {
   border: solid 1pt black;
}
```

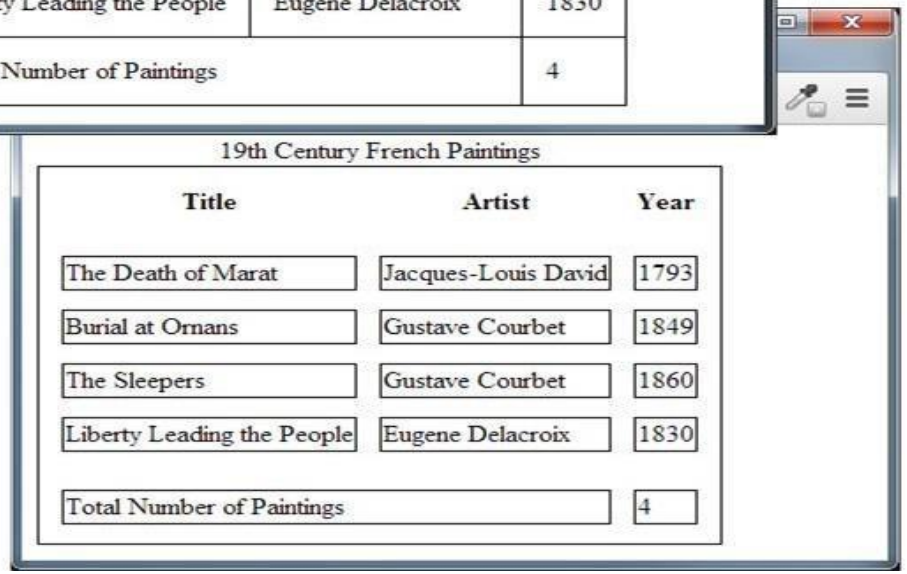



```
table {
   border: solid 1pt black;
   border-collapse: collapse;
}
td {
   border: solid 1pt black;
}
```

Styling Tables
Padding and spacing



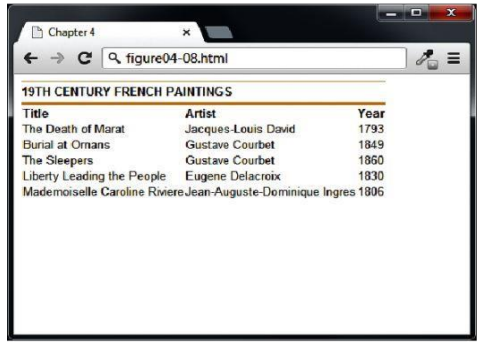

```
table {
    border: solid 1pt black;
    border-collapse: collapse;
}
td {
    border: solid 1pt black;
    padding: 10pt;
}
```



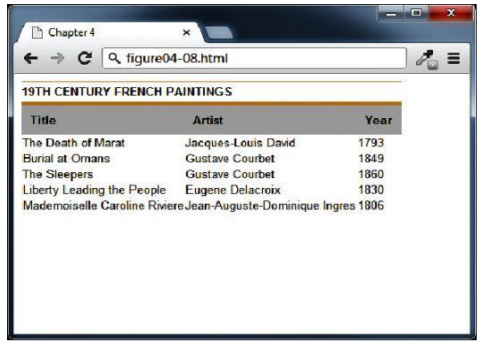

```
table {
    border: solid 1pt black;
    border-spacing: 10pt;
}
td {
    border: solid 1pt black;
}
```

Styling Tables

Examples




```
table {
   font-size: 0.8em;
   font-family: Arial, Helvetica, sans-serif;
   border-collapse: collapse;
   border-top: 4px solid #DCA806;
   border-bottom: 1px solid white;
   text-align: left;
}
caption {
   font-weight: bold;
   padding: 0.25em 0 0.25em 0;
   text-align: left;
   text-transform: uppercase;
   border-top: 1px solid #DCA806;
}
```



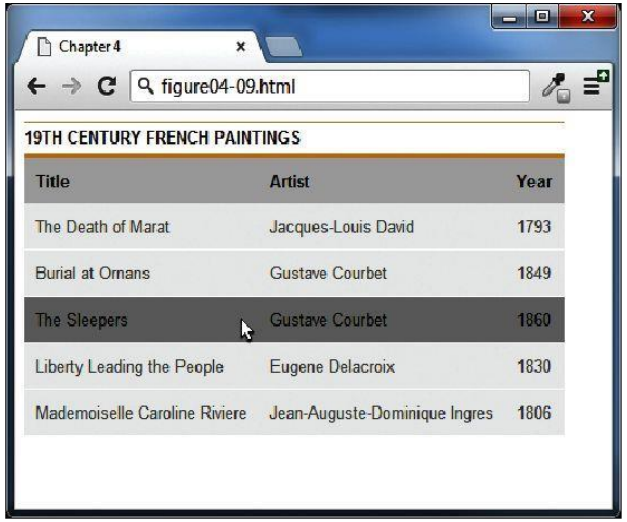

```
thead tr {
   background-color: #CACACA;
}
th {
   padding: 0.75em;
}
```




```
tbody tr {
   background-color: #F1F1F1;
   border-bottom: 1px solid white;
   color: #6E6E6E;
}
tbody td {
   padding: 0.75em;
}
```

Nth-Child

Nifty Table styling tricks: hover effect and zebra-stripes




```
tbody tr:hover {
    background-color: #9e9e9e;
    color: black;
}
```



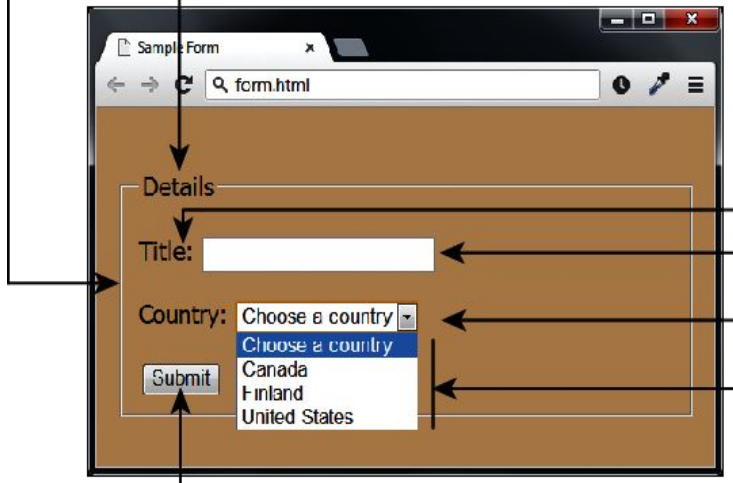

```
tbody tr:nth-child(odd) {
    background-color: white;
}
```

INTRODUCING FORMS

HTML Forms
Richer way to interact with server

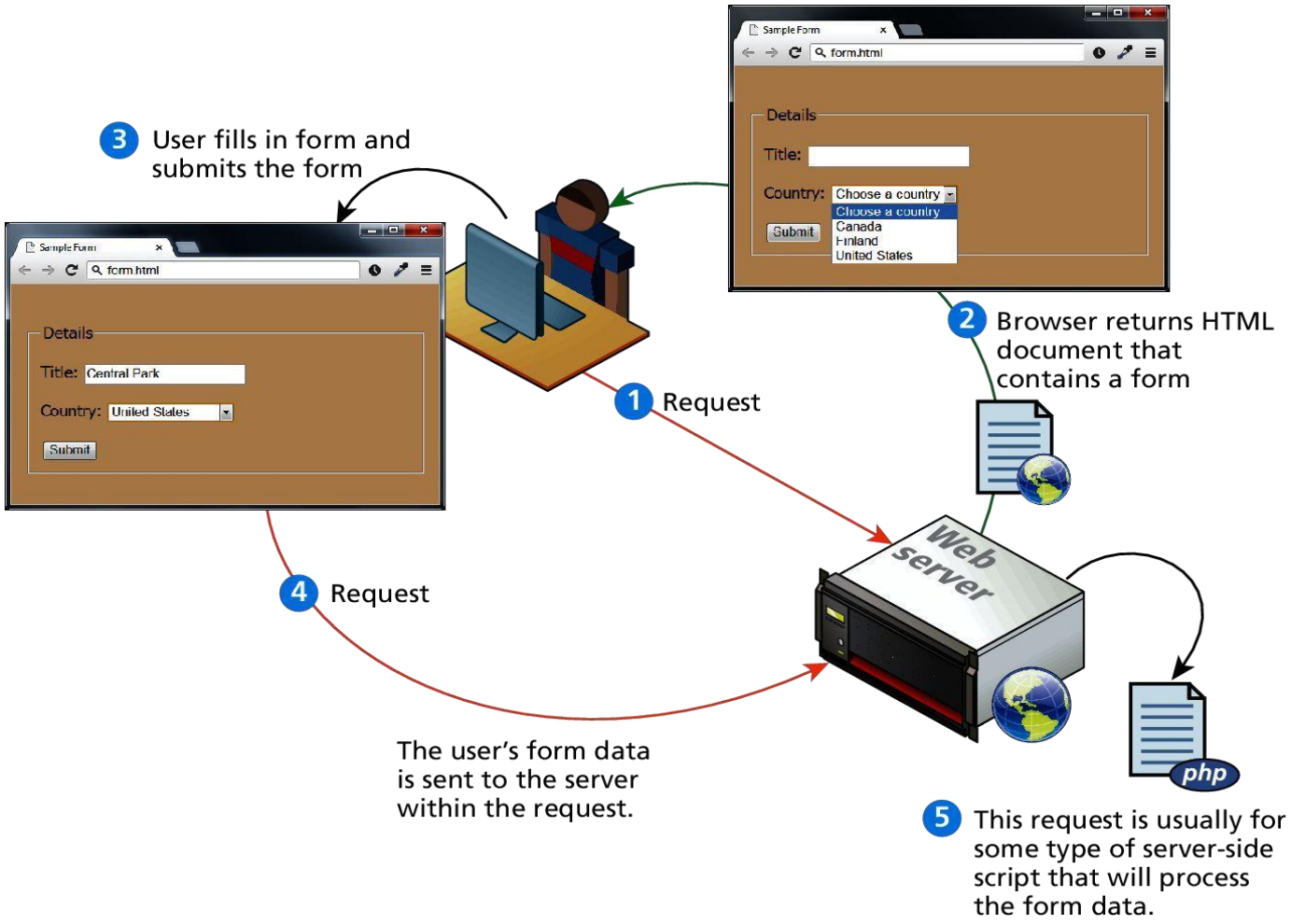
- **Forms** provide the user with an alternative way to interact with a
- web server.
  - Forms provide rich mechanisms like:
    - Text input
    - Password input
    - Options Lists
    - Radio and check boxes

Form Structure




```
<form method="get" action="process.php">
 <fieldset>
  <legend>Details</legend>
  <p>
   <label>Title: </label>
   <input type="text" name="title" />
  </p>
  <p>
   <label>Country: </label>
   <select name="where">
    <option>Choose a country</option>
    <option>Canada</option>
    <option>Finland</option>
    <option>United States</option>
   </select>
  </p>
  <input type="submit" />
 </fieldset>
</form>
```

How forms interact with servers





The HTML <form> element is used to create an HTML form for user input:

<form>
.
form elements
.
</form>

The <input> Element

- The HTML <input> element is the most used form element.
- An <input> element can be displayed in many ways, depending on the type attribute.

Type	Description
<input type="text">	Displays a single-line text input field
<input type="radio">	Displays a radio button (for selecting one of many choices)
<input type="checkbox">	Displays a checkbox (for selecting zero or more of many choices)
<input type="submit">	Displays a submit button (for submitting the form)
<input type="button">	Displays a clickable button

Query Strings

You may be wondering how the browser "sends" the data to the server.

This occurs via an HTTP request.

The browser packages the user's data input into something called a query string.

A **query string** is a series of name=value pairs separated by ampersands (the **&** character).

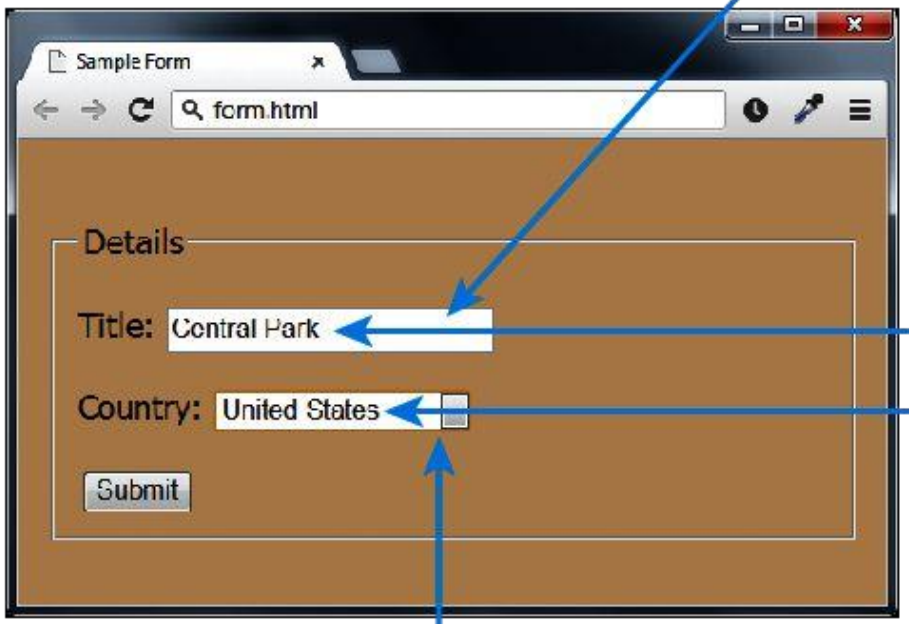
The values in the query string are the data entered by the user.

Figure illustrates how the form data (and its connection to form elements) is packaged into a query string

Query Strings

At the end of the day, another string

```
<input type="text" name="title" />
```




```
<select name="where">
```

```
title=Central+Park&where=United+States
```

<form> element

- Two essential features of any form, namely the **action** and the
- **method** attributes.
  - The **action** attribute specifies the URL of the server-side
  - resource that will process the form data
  - The **method** attribute specifies how the query string data will be transmitted from the browser to the server.
    - GET
    - POST

GET vs POST

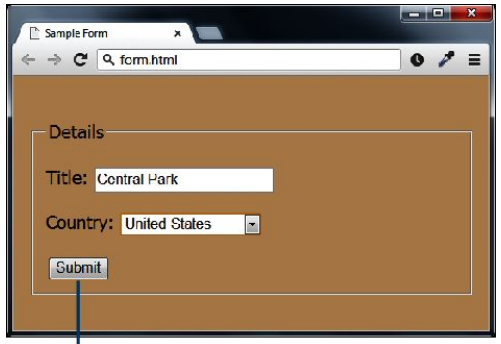
GET

- Data can be clearly seen in the address bar.
- Data remains in browser history and cache.
- Data can be bookmarked
- Limit on the number of characters in the form data returned.

POST

- Data is hidden from user.
- Data can contain binary data.
- Submitted data is not stored in cache, history, or bookmarks.
- Limit on the number of characters in the form data returned.

GET vs POST




```
<form method="get" action="process.php">
```

```
GET /process.php?title=Central+Park&where=United+States http/1.1
```

querystring

```
<form method="post" action="process.php">
```

```
POST /process.php http/1.1
Date: Sun, 20 May 2012 23:59:59 GMT
Host: www.mysite.com
User-Agent: Mozilla/4.0
Content-Length: 47

title=Central+Park&where=United+States
```

HTTP Header

querystring

FORMS CONTROL ELEMENTS

Form-Related HTML Elements

Type	Description
<button>	Defines a clickable button.
<datalist>	An HTML5 element form defines lists to be used with other form elements.
<fieldset>	Groups related elements in a form together.
<form>	Defines the form container.
<input>	Defines an input field. HTML5 defines over 20 different types of input.
<label>	Defines a label for a form input element.
<legend>	Defines the label for a fieldset group.
<option>	Defines an option in a multi-item list.
<optgroup>	Defines a group of related options in a multi-item list.
<select>	Defines a multi-item list.
<textarea>	Defines a multiline text entry box.

Text Input Controls

Type	Description
text	Creates a single line text entry box. **<input type="text" name="title" />**
Textarea	Creates a multiline text entry box. **<textarea rows="3" ... />**
password	Creates a single line text entry box for a password **<input type="password" ... />**
Search	Creates a single-line text entry box suitable for a search string. This is an HTML5 element. **<input type="search" … />**
Email	Creates a single-line text entry box suitable for entering an email address. This is an HTML5 element. **<input type="email" … />**
Tel	Creates a single-line text entry box suitable for entering a telephone. This is an HTML5 element. **<input type="tel" … />**
url	Creates a single-line text entry box suitable for entering a URL. This is an HTML5 element. **<input type="url" … />**

Classic

Text Input Controls

```
<input type="text" ... />
```

Text:

```
<textarea>
  enter some text
</textarea>
```

```
<textarea placeholder="enter some text">
</textarea>
```

TextArea: enter some text

TextArea: Enter some text

```
<input type="password" ... />
```

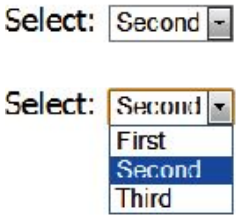
Password:

Password: ••••

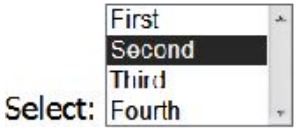
Choice Controls

- Select Lists
- Chose an option, any option.
  - **<select>** element is used to create a multiline box for
  - Selecting one or more items
  - The options are defined using the **<option>** element
  - Can be hidden in a dropdown or multiple rows of the list can be visible
  - Option items can be grouped together via the **<optgroup>** element.

Select Lists
Select List Examples




```
<select name="choices">
  <option>First</option>
  <option selected>Second</option>
  <option>Third</option>
</select>
```




```
<select size="3" ... >
```

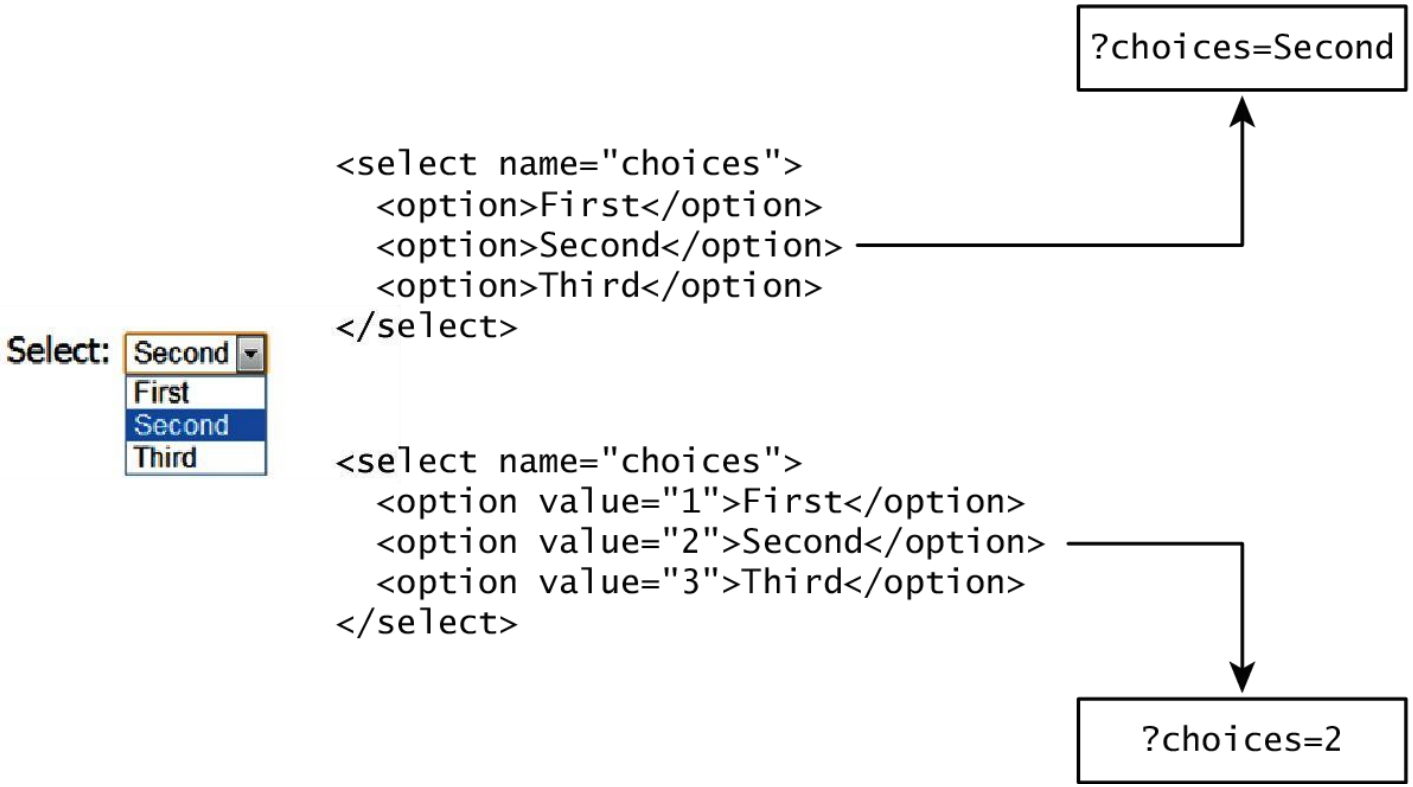



```
<select … >
 <optgroup label="North America">
   <option>Calgary</option>
   <option>Los Angeles</option>
 </optgroup>
 <optgroup label="Europe">
   <option>London</option>
   <option>Paris</option>
   <option>Prague</option>
 </optgroup>
</select>
```

Which Value to send Select Lists Cont.

- The **value** attribute of the <option> element is used to specify
- what value will be sent back to the server.

- The value attribute is optional; if it is not specified, then the text within the container is sent instead



Radio Buttons

Radio buttons are useful when you want the user to select a single item from a small list of choices and you want all the choices to be visible

- radio buttons are added via the **<input type="radio">** element
- The buttons are mutually exclusive (i.e., only one can be chosen)
- by sharing the same name attribute
- The checked attribute is used to indicate the default choice
- the value attribute works in the same manner as with the <option>
- element




```
<input type="radio" name="where" value="1">North America<br/>
<input type="radio" name="where" value="2" checked>South America<br/>
<input type="radio" name="where" value="3">Asia
```

Checkboxes

Checkboxes are used for getting yes/no or on/off responses
from the user.

- checkboxes are added via **the <input type="checkbox”>** Element
- You can also group checkboxes together by having them share the same name attribute
- Each checked checkbox will have its value sent to the server
- Like with radio buttons, the checked attribute can be used to set the default value of a checkbox

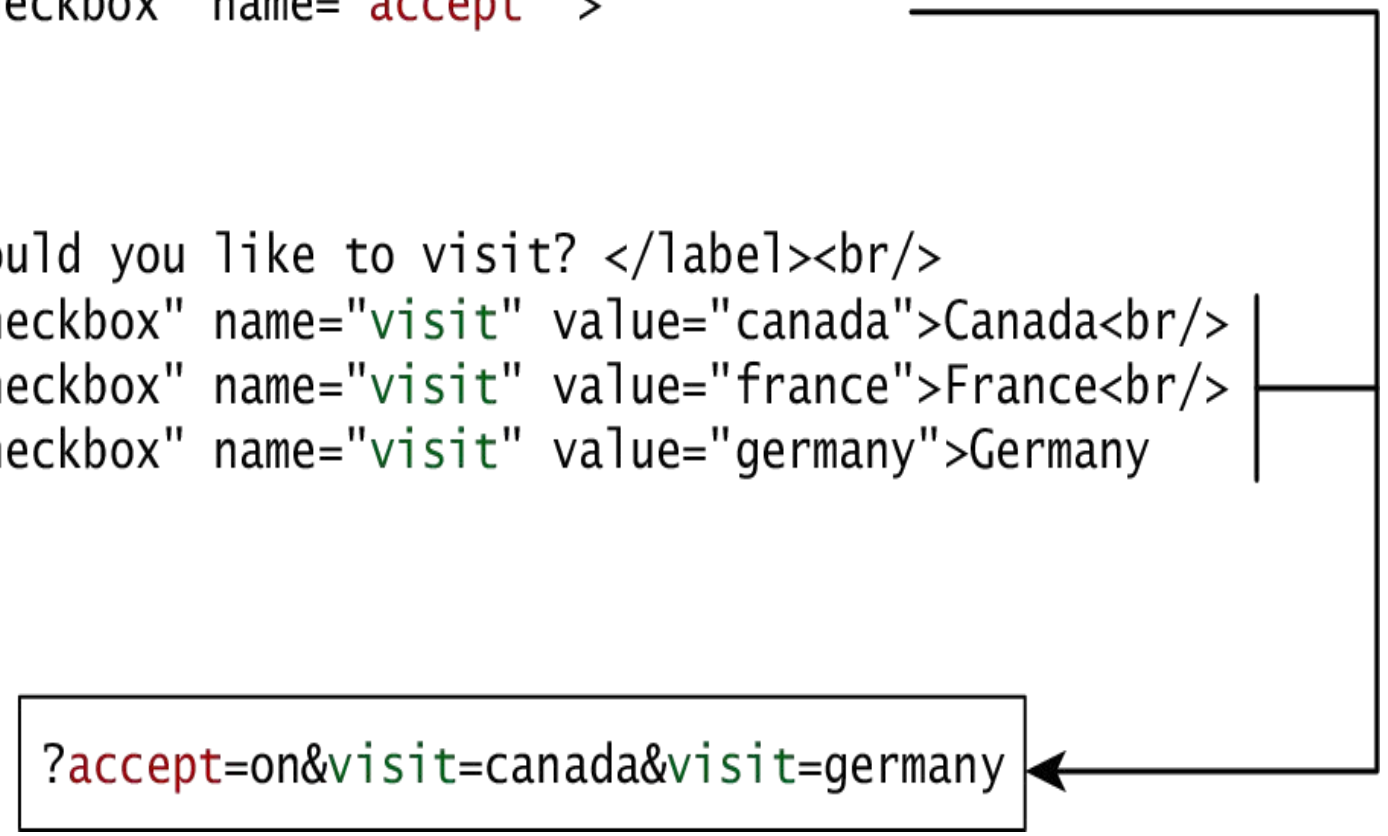
Checkboxes

I accept the software license ☑

```
<label>I accept the software license</label>
<input type="checkbox" name="accept" >
```

Where would you like to visit?
- ☑ Canada
- ☐ France
- ☑ Germany

```
<label>Where would you like to visit? </label><br/>
<input type="checkbox" name="visit" value="canada">Canada<br/>
<input type="checkbox" name="visit" value="france">France<br/>
<input type="checkbox" name="visit" value="germany">Germany
```

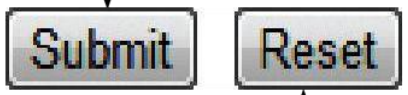


Button Controls

TYPE	DESCRIPTION
<input type="submit">	Creates a button that submits the form data to the server.
<input type="reset">	Creates a button that clears any of the user's already entered form data.
<input type="button">	Creates a custom button. This button may require Javascript for it to actually perform any action.
<input type="image">	Creates a custom submit button that uses an image for its display.
<button>	Creates a custom button. The <button> element differs from <input type="button"> in that you can completely customize what appears in the button; using it, you can, for instance, include both images and text, or skip server-side processing entirely by using hyperlinks. You can turn the button into a submit button by using the type="submit" attribute.

Button Controls

```
<input type="submit"  />
```

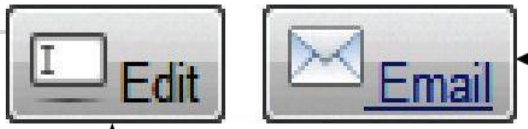



```
<input type="reset"  />
```

```
<input type="button" value="Click Me" />
```




```
<input type="image" src="appointment.png" />
```

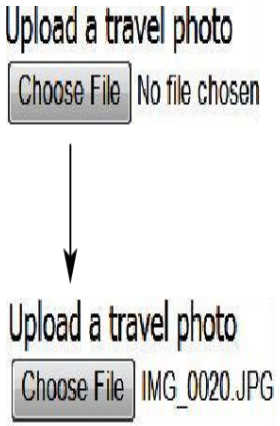



```
<button>
   <a href="email.html">
      <img src="images/email.png" alt=""/>
      Email
   </a>
</button>
```

```
<button type="submit"  >
   <img src="images/edit.png" alt=""/>
   Edit
</button>
```

Specialized Controls

- I'm so special
  - **<input type=hidden>**
  - **<input type=file>**




```
<form method="post" enctype="multipart/form-data" ... >
   ...
   <label>Upload a travel photo</label>
   <input type="file" name="photo" />
   ...
</form>
```

Number and Range

Typically input values need be **validated**. Although server side validation is required, optional client side pre-validation is good practice.

The number and range controls Added in HTML5 provide a way to input numeric values that eliminates the need for JavaScript numeric validation!!!

Number and Range

Rate this photo:
2

```
<label>Rate this photo: <br/>
<input type="number" min="1" max="5" name="rate" />
```

Grumpy Ecstatic

```
Grumpy
<input type="range" min="0" max="10" step="1" name="happiness" />
Ecstatic
```

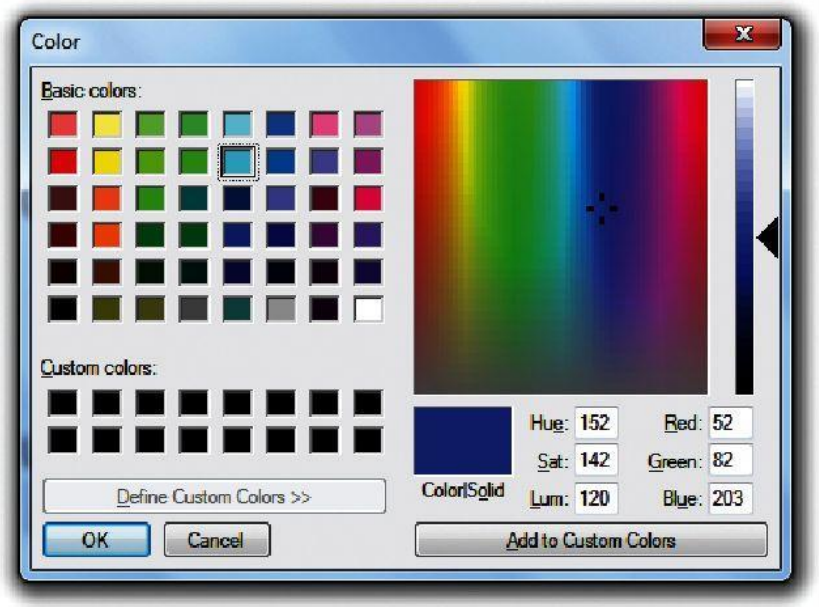
Rate this photo:

Grumpy Ecstatic

Controls as they appear in browser
that doesn't support these input types

Color

Background Color:




```
<label>Background Color: <br/>
<input type="color" name="back" />
```

Background Color:

Control as it appears in browser that doesn't support this input type

Date and Time Controls



- Dates and times often need validation when gathering this information from a regular text input control.
- From a user's perspective, entering dates can be tricky as well: you probably have wondered at some point in time when entering a date into a web form, what format to enter it in, whether the day comes before the month, whether the month should be entered as an abbreviation or a number, and so on.

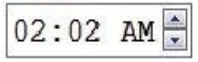
HTML5 Date and Time Controls

Date:




```
<label>Date: <br/>
<input type="date" ... />
```

Time:




```
<input type="time" ... />
```

DateTime:




```
<input type="datetime" ... />
```

DateTime Local:




```
<input type="datetime-local" ... />
```

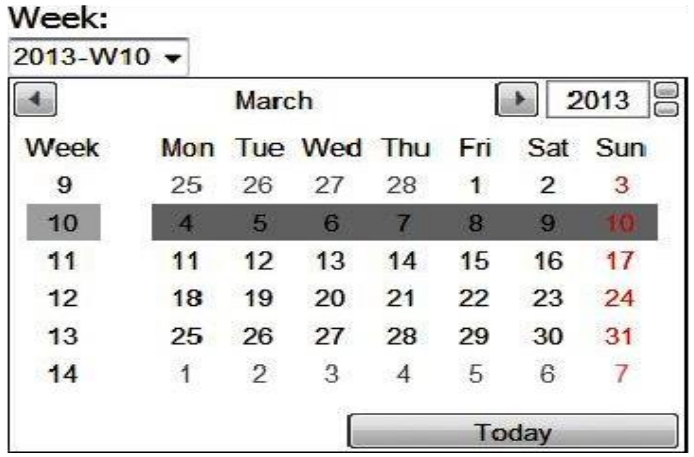
HTML5 Date and Time Controls

Month:




```
<input type="month" ... />
```

Week:




```
<input type="week" ... />
```

HTML Controls

Type	Description
date	Creates a general date input control. The format for the date is "yyyy-mm-dd".
time	Creates a time input control. The format for the time is "HH:MM:SS", for hours:minutes:seconds.
datetime	Creates a control in which the user can enter a date and time.
datetime-local	Creates a control in which the user can enter a date and time without specifying a time zone.
month	Creates a control in which the user can enter a month in a year. The format is "yyyy-mm".
week	Creates a control in which the user can specify a week in a year. The format is "yyyy-W##".

Other Controls

- The <progress> and <meter> elements can be used to
- provide feedback to users,
  - but requires JavaScript to function dynamically.
- The <output> element can be used to hold the output
- from a calculation.
- The <keygen> element can be used to hold a private key for public-key encryption

XHTML

- The goal of XHTML with its strict rules was to make page rendering more predictable by forcing web authors to create web pages without syntax errors.

- Following is the xml declaration element, which should be the first line of every XHTML document.

- <?xml version="1.0" encoding="UTF-8"?>

XHTML

- <!DOCTYPE> is mandatory
- The xmlns mandatory
- <html>, <head>, <title>, and <body> are mandatory
- Elements must always be properly nested
- Elements must always be closed
- Elements must always be in lowercase
- Attribute names must always be in lowercase
- Attribute values must always be quoted

XHTML documents must include Four tags

1)<html>

2)<head>

3)<title>

4)<body>

XHTML

The XML-based syntax rules for XHTML are pretty easyto follow.

The main rules are:

- lowercase tag names,
- attributes always within quotes,
- and all elements must have a closing element (or be self- closing).

XHTML

Two versions

To help web authors, two versions of XHTML were created:

XHTML 1.0 Strict and **XHTML 1.0 Transitional**.

- The **strict** version was meant to be rendered by a browser using the strict syntax rules and tag support described by the W3C XHTML 1.0 Strict specification.
- The **transitional** recommendation is a more forgiving flavor of XHTML, and was meant to act as a temporary transition to the eventual global adoption of XHTML Strict.

XML



XML stands for **eXtensible Markup Language**.



It is a **markup language** designed to store and transport data.



XML is **both human-readable and machine-readable**.



Unlike HTML, which focuses on how data looks, XML focuses on **what data is**.



XML allows you to define your own **custom tags** to describe the structure and meaning of data.



It is **platform-independent** and widely used for data interchange between systems.

Uses of XML

- **Data Storage:** Storing structured data in files or databases.
- **Data Transport:** Sharing data across different systems, platforms, or applications.
- **Configuration Files:** Many software applications use XML to store configuration settings.
- **Web Services:** XML is a foundation for protocols like SOAP (Simple Object Access Protocol) used in web services.
- **RSS Feeds:** XML formats content updates in web feeds.
- **Document Formats:** Formats like Microsoft Office Open XML (used in Word, Excel) and SVG use XML syntax.
- **Interoperability:** Enables communication between heterogeneous systems in enterprise applications.

XML Document

```
<root>
  <child>
    <subchild>.....</subchild>
  </child>
</root>
```

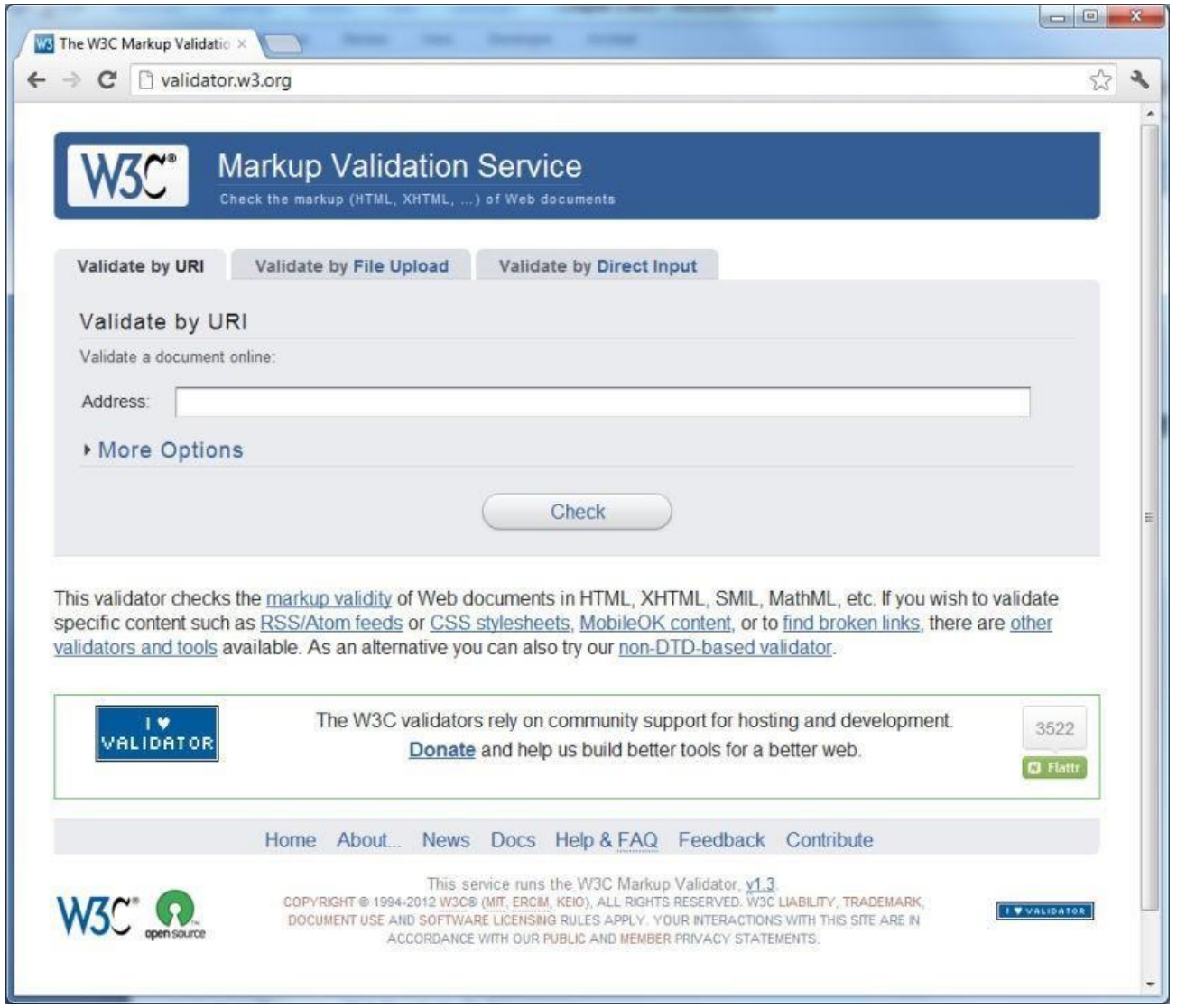
```
<?xml version="1.0" encoding="UTF-8"?>
<note>
  <to>Yash</to>
  <from>Radhika</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
</note>
```

Aspect	HTML	XHTML
Introduction	HTML - HyperText Markup Language	XHTML - Extensible HyperText Markup Language
Developed by	W3C & WHATWG	World Wide Web Consortium
Versions	HTML2, HTML3.2, HTML4.0, HTML5	XHTML1, XHTML1.1, XHTML2, XHTML5
Application	Standard Generalized Markup Language (SGML)	Application of XML
Filename extension	.html, .htm	.xhtml, .xht, .xml, .html, .htm
Extended from	SGML	XML, HTML
Type	text/html	application/xhtml+xml
Stands for	HyperText Markup Language	Extensible HyperText Markup Language
Type of format	Document file format	Markup language
Function	Here web pages are written in HTML.	It is a very strict version of HTML and based on XML.
Nature	Flexible framework requiring lenient HTML parser.	A restrictive subset of XML and needs to be parsed with standard XML parsers.
Origin	HTML was created by Tim Berners-Lee	(W3C) World Wide Web Consortium Recommendation in 2000.

Validators

How to ensure your pages follow a standard

A key part of the standards movement in the web development community of the 2000s was the use of **HTML Validators** as a means of verifying that a web page's markup followed the rules for XHTML transitional or strict.



How about an example

Only if you have an internet connection



- Open a web browser to the W3C validator and find a few websites to test.

- Type the URL into the bar, and you can check if the home page is valid against various standards (or auto-detect)

XHTML 2.0 and WHATWG Where did it go?

- In the mid 2000s, XHTML 2.0 proposed a revolutionary and substantial change to HTML.
  - backwards compatibility with HTML and XHTML 1.0 was dropped.
  - Browsers would become significantly less forgiving of invalid
  - markup.
  - At around the same time, a group of developers at Opera and Mozilla formed the **WHATWG** (Web Hypertext Application Technology Working Group) group within the W3C.
  - This group was not convinced that the W3C's embrace of XML and its abandonment of backwards-compatibility was the best way forward for the web.

XHTML versus HTML5

